

SIMULINK动态仿真集成环境

SIMULINK动态仿真集成环境

主要内容

- **Simulink简介**
- **Simulink动态系统建模与仿真原理**
- **S-Funcition**
- 命令行仿真技术
- 综合实例

一 Simulink简介

1.1 Simulink简介

Simulink是MATLAB的重要组成部分，提供建立系统模型、选择仿真参数和数值算法、启动仿真程序对该系统进行仿真、设置不同的输出方式来观察仿真结果等功能。



Simu+Lab1990 → **Simu+Link1992**

Simu (仿真) 和Link (模型连接)

1.1.2Simulink的功能及特点

1)功能

- 它是一种基于MATLAB的框图设计环境，被广泛应用于线性系统、非线性系统、数字控制及数字信号处理等领域的建模和仿真中
- 可以二次开发，在Simulink基础之上构建其他产品，从而扩展了Simulink多领域建模功能，
- 可以直接访问MATLAB大量的工具来进行算法研发、仿真分析和可视化、批处理脚本的创建、建模环境的定制以及信号参数和测试数据的定义
- Simulink提供了交互式图形化环境和可定制模块库来对其进行设计、仿真、执行和测试

2) 特点

- 为建立各种各样的系统模型，Simulink除提供了一些基本库之外，针对特定领域还提供了丰富的可扩充的预定义模块库
- 交互式的图形编辑器来组合和管理直观的模块图
- 以设计功能的层次性来分割模型，实现对复杂设计的管理
- 通过Model Explorer 导航、创建、配置、搜索模型中的任意信号、参数、属性，生成模型代码
- 提供API用于与其他仿真程序的连接或与手写代码集成
- 使用Embedded MATLAB™ 模块在Simulink和嵌入式系统执行中调用MATLAB算法

续. 特点

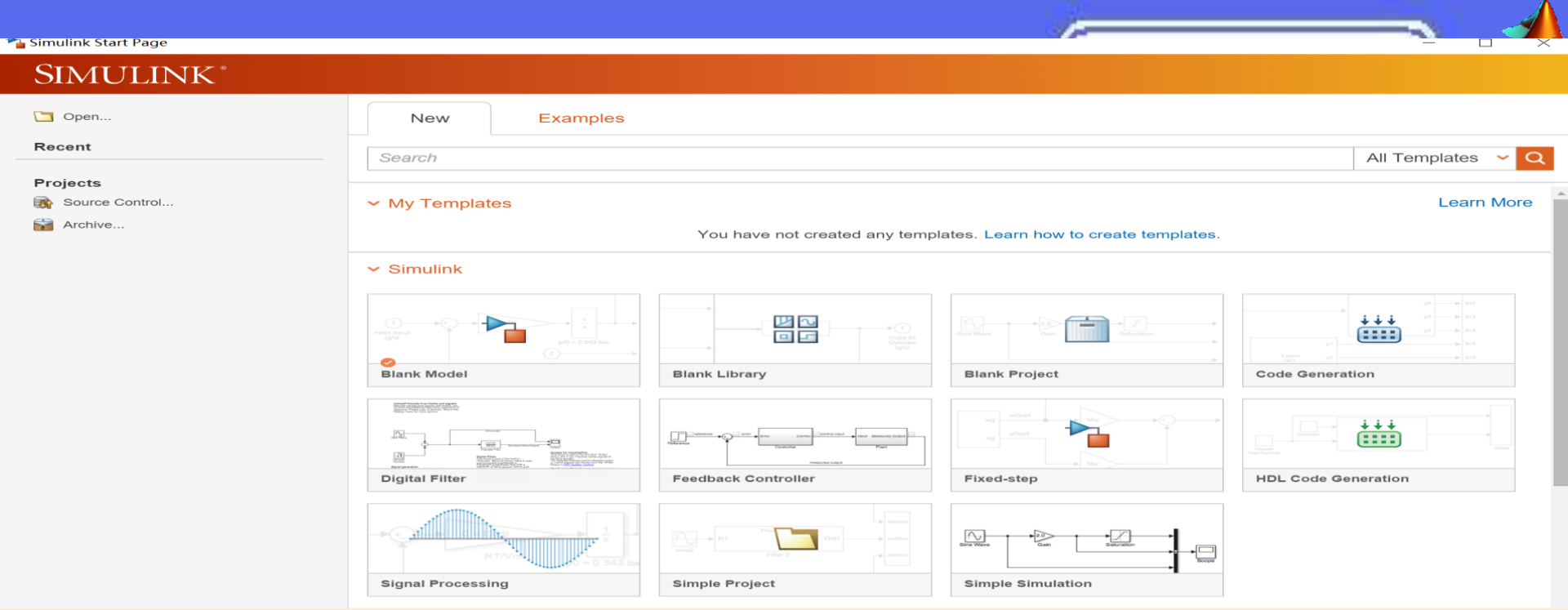
- 使用定步长或变步长运行仿真，根据仿真模式来决定以解释性的方式或以编译C代码的形式来运行模型
- 图形化的调试器(Debugger)和剖析器(Profiler)来检查仿真结果，诊断设计的性能和异常行为
- 可访问MATLAB，从而对结果进行分析与可视化，定制建模环境，定义信号参数和测试数据
- 模型分析和诊断工具来保证模型的一致性，确定模型中的错误
- 使得用户可以把更多的精力投入到系统模型的构建而非编程上。

1.2 Simulink的启动与退出

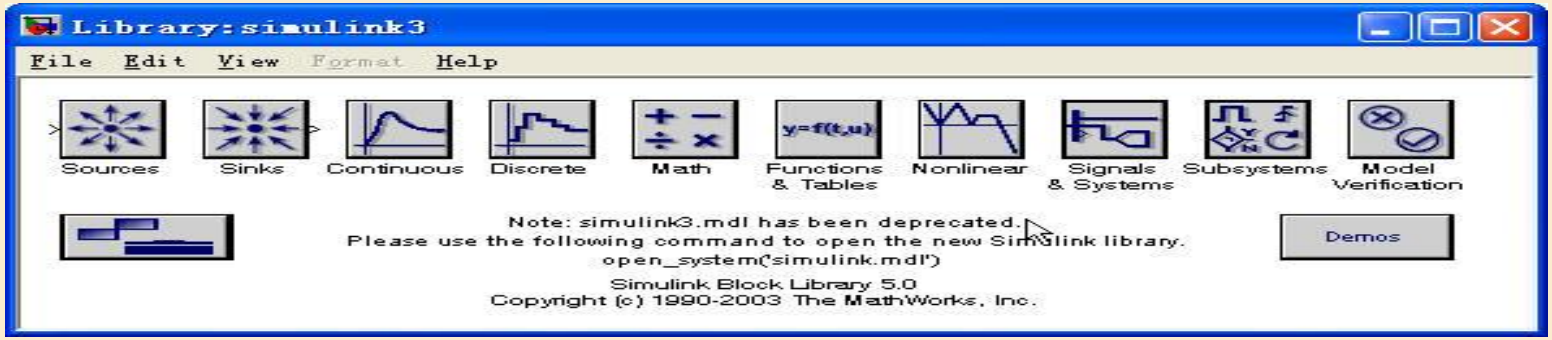
1. Simulink的启动

- 在MATLAB的命令窗口输入simulink
- **使用主窗口工具栏按钮启动Simulink，即用鼠标单击MATLAB工具栏中的Simulink 按钮**
- **在MATLAB的命令窗口中直接键入“Simulink3”，同样会出现一个用图标形式显示的Library**

Simulink启动后会显示Simulink startpage



Simulink库浏览器 (Simulink Library Browser)窗口



Simulink3模块库窗口

2．Simulink的退出

为了退出Simulink，只要关闭所有模型编辑窗口和Simulink模块库浏览器窗口即可。

二、Simulink仿真原理

1. 系统、模型与仿真

● 系统

是指由相互联系、相互作用的实体集合而成，并表现出某些特定功能的一个整体。组成系统的实体之间相互作用而引起的实体属性的变化，通常用状态变量来描述。

研究系统主要研究系统的动态变化。除了研究系统的实体属性活动外，还需要研究影响系统活动的外部条件—环境。

Simulink中搭建的系统有三种类型：

离散系统：系统的操作和状态变化仅在离散时刻产生

连续系统：具有连续的输入与输出，系统输出在时间上连续变化，变化的间隔为无穷小量，而非仅在离散的时刻采样取值

混合系统：混合系统就是连续系统和离散系统的混合，系统模型中既有连续状态，又有离散状态。

● 模型

系统模型是对实际系统的一种抽象，是对系统本质(或是系统的某种特性)的一种描述。

在计算机上研究系统的动态特性，就需要建立数学模型

数学模型按照状态变化可分为：动态模型和静态模型

描述系统状态变化过程的数学模型称为动态模型。

Simulink中的模型主要是指仿真系统数学模型，它是一种适合在计算机上演算的模型，主要是指根据计算机的运算特点、仿真方式、计算方法、精度要求将原始系统数学模型转换为计算机程序。

● 仿真

仿真是以相似性原理、控制论、信息技术及相关领域的有关知识为基础，以计算机和各种专用物理设备为工具，借助系统模型对真实系统进行试验研究的一门综合性技术。它利用物理或数学方法来建立模型，类比模拟现实过程或者建立假想系统，以寻求过程的规律，研究系统的动态特性，从而达到认识和改造实际系统的目的。

物理仿真：

研制某些实体模型，使之重现系统的各种状态

数学仿真：

用数学语言去表述一个系统，并编制程序在计算机上对实际系统进行研究的过程

对系统进行仿真时，除了模型外还需要仿真算法，仿真软件

2. Simulink仿真原理

- 微分方程是描述动态系统最常用的数学工具，也是很多科学与工程领域数学建模的基础。由于一般的非线性微分方程是没有解析解的，故需用数值解的方式求解。
- Simulink中系统的仿真主要就是利用解各类微分方程的数值解来实现的。连续的求解器可以计算连续或混合系统，而离散的求解器，则只能解离散系统。

Simulink系统模型的仿真主要包含两个阶段：

1）初始化阶段

- 每个模块的所有参数都传递给MATLAB进行求值，得到的数值作为实际的参数使用。
- 展开模型的层次结构，每个子系统被它们所包含的模块替代，带有触发和使能模块的子系统被视为原子单元进行处理。
- 检查信号的宽度和模块的连接情况，提取状态和输入、输出依赖关系方面的信息，确定模块的更新顺序。
- 确定状态的初值和采样时间。

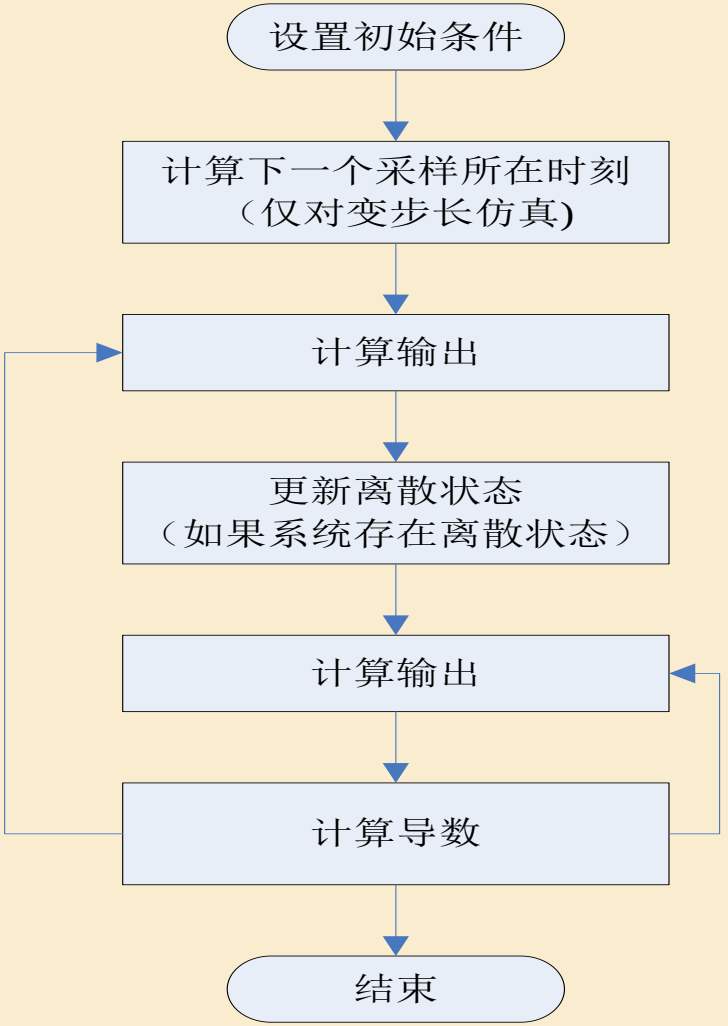
2）运行阶段

初始化之后，仿真进入运行阶段。仿真是由求解器控制的，它计算模块的输出，更新离散状态，计算连续状态。

Simulink仿真过程



系统和求解器交互作用



Simulink仿真过程

3. 仿真求解器

- Simulink主要的特征是状态的更新，涉及到对状态方程中微分或差分方程的求解，因而采用的求解器，也就是解方程的算法也就至关重要，直接决定了仿真的效果和精度。
- 系统模型搭建完成之后，应该设置模块参数和仿真操作参数，然后就可以对其进行各种各样的仿真，以测试系统的性能。

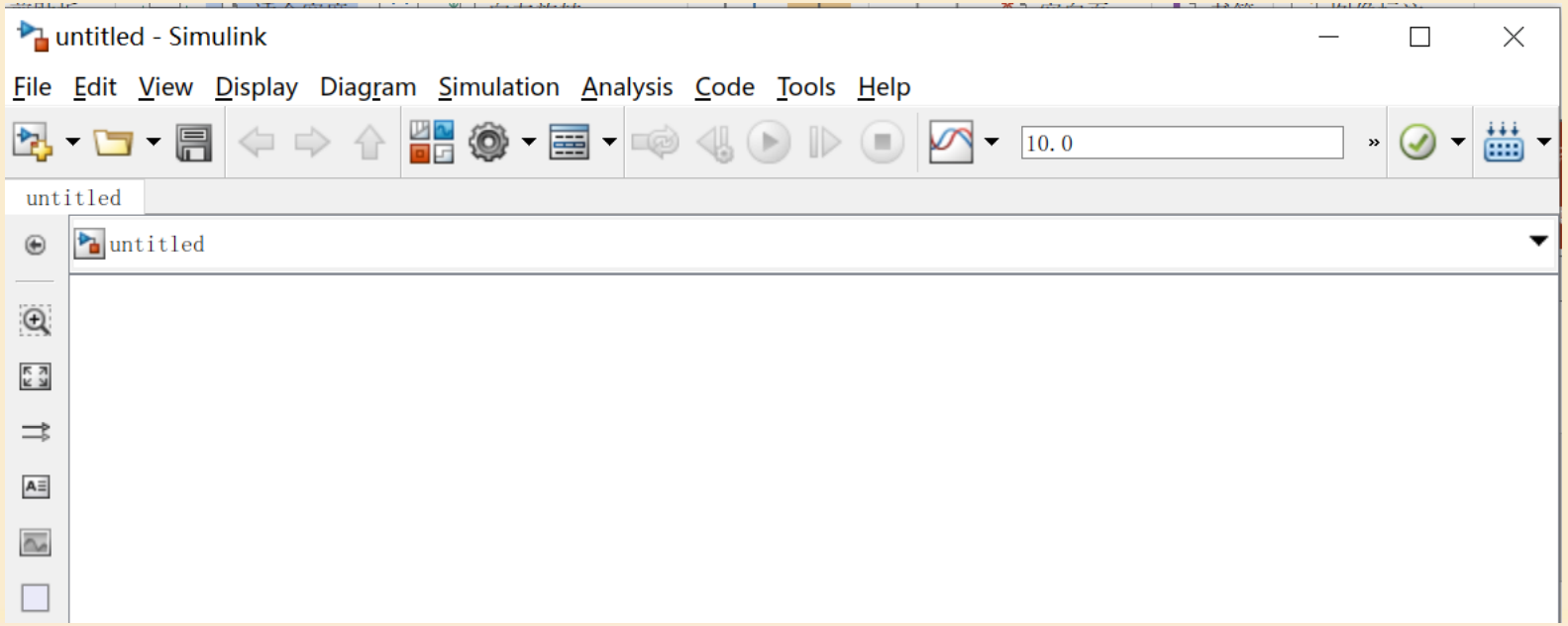
以R2016B为例

启动MATLAB，并按照下图红色箭头所示步骤操作就可以打开一个模型的仿真求解器设置界面，根据系统模型的类型和特点进行相应的求解器参数设置，来进行系统仿真。


MATLAB R2016b
主页
绘图
APP
编辑器
发布
视图
搜索文档
新建脚本
新建
打开
查找文件
比较
导入数据
保存工作区
新建变量
打开变量
清除工作区
分析代码
运行并计时
清除命令
Simulink
布局
预设
设置路径
Parallel
附加功能
帮助
社区
请求支持
了解 MATLAB
文件
变量
代码
SIMULINK
环境
资源

在启动Simulink模块库浏览器后再单击其工具栏中的New-Blank model命令按钮，会弹出名字为untitled-Simulink的模型编辑窗口。

在主菜单中，选择File菜单中New菜单项的Blank Model命令，也可打开模型编辑窗口。利用模型编辑窗口，可以通过鼠标的拖放操作创建一个模型。



4. 过零检测

在动态系统的仿真过程中，所谓过零，是指系统模型中的信号或系统模块特征的某种改变。

这种特征的改变包括：

- 信号在上一个时间步改变了符号（包括变为零和离开零）。
- 模块在上一个时间步改变了模式（如积分器进入了饱和区段）。

关闭系统仿真参数设置中的过零事件检测，可以使动态系统的仿真速度得到很大的提高。但可能会引起系统仿真结果的不精确，甚至出现错误结果。关闭系统过零对**Hit Crossing**零交叉模块并无影响。对于离散模块及其产生的离散信号不需要进行过零检测。

- 过零检测通过在系统和求解器之间建立对话的方式工作。对话包含的一个内容是事件通知，即系统告知求解器在前一时间步发生了一个事件。过零是一个重要的事件，表征系统中的不连续性，例如响应中的跳变。如果仿真过程中，对过零不进行检测，可能会导致不准确的仿真结果。当采用变步长求解器时，Simulink能够检测到过零。使用固定步长的求解器时，Simulink不检测过零。当一个模块通知系统前一时间步发生了过零，变步长求解器就会缩小步长，即使绝对误差和相对误差是在可接受的范围内。缩小步长的目的是判定过零事件发生的准确时间。当然，这样会降低仿真的速度，但这样做，对于有些模块来讲是至关重要和必要的。因为这些模块的输出可能表示了一个物理值，它专属于自己的事件通知，而且可能与不止一个类型的事件发生关联。

产生过零通知的模块表

模块名	过零描述
Abs	当输入信号的上升沿或者下降沿检测一次是否过零
Backlash	当上限阈值达到时，检测一次过零，到达下限阈值时，也检测一次过零，共两次
Compare To Constant	当信号等于一个常量时，检测过零
Compare To Zero	当信号等于零时，检测过零
Dead Zone	当输入信号进入死区时，检测过零，当输入信号脱离死区时，也检测过零
Enable	如果使能口在一个子系统模块的内部，则提供检测过零的能力
From Workspace	当输入信号在上升沿或者下降沿有不连续时，就检测过零
If	当满足If条件时，检测过零
Integrator	如果存在复位口，当复位时，检测过零，如果输出受限，在下面三种情况下，检测过零： 1，达到上饱和限，2，达到下饱和限，3，离开饱和时
MinMax	对输出向量的每个元素，当输入信号是新的最小或最大值时，检测过零
Relational Operator	当相应的运算关系为真时，检测过零
Relay	如果relay处于关断状态，检测开关点的过零，如果relay处于打开状态，检测关断点的过零
Saturation	达到饱和上限或离开时，检测过零，达到饱和下限或离开时，也检测过零
Sign	当输入信号交叉通过零时，检测过零
Signal Builder	当输入信号在上升或者下降方向有不连续时，检测过零
Step	在产生阶跃的时刻，检测过零
Switch	当开关条件发生时，检测过零
Switch Case	当case条件满足时，检测过零
Trigger	如果触发口在一个子系统模块的内部，则提供检测过零的能力
Enabled and Triggered Subsystem	在一个子系统模块的内部，对使能口和触发口分别进行过零检测
Hit Crossing	当输入穿过零点时产生一个过零，可以用来为不带过零能力的模块提供过零检测能力

5. 代数环

- 在使用Simulink的模块库建立动态系统的模型时，有些模块的输入端口具有直接馈通特性。
- 所谓直接馈通(Direct feedthrough)是指模块的输出直接依赖于模块的输入，如果模块的输出方程中包含输入，则它具备直接馈通特性。
- 在Simulink模型中，将带有直接馈通特性的各模块串成一个回路会导致一个代数环。
- 在一个代数环中，由于模块之间是相互依赖的，所有的模块都要求在同一时刻计算输出，这与通常的仿真顺序概念相抵触。

✓ 具有直接馈通特性的模块表

模块名	模块描述	模块名	模块描述
Math Function	数学函数	Transfer Fcn	传递函数
Gain	增益	Sum	求和
Product	乘法	Zero-Pole	零极点
State-Space	状态空间（具有非零D阵）	Integrator	积分（初始条件端口）

- **具有代数环的模型，仿真得到的结果可能不正确。一般在用Simulink搭建系统模型之前，应尽量通过手工方法对方程求解，以去掉代数环。**
- **Simulink自带一个内置的代数环求解器，可以解决代数环问题，也可以使用Simulink/Math库中的Algebraic Constraint模块对动态系统数学方程进行求解。**

二 系统仿真模型

2.1 Simulink的基本模块

Simulink的模块库提供了大量模块。 **MathWorks公司提供的面向特定领域的专业模块库，它是Simulink基本功能的扩展，极大地方便了用户搭建特定应用领域的模型**

同样，在编辑器工具栏上单击  ，将打开Simulink基本模块库窗口。单击其中的子模块库图标，打开子模块库，找到仿真所需要的基本模块。

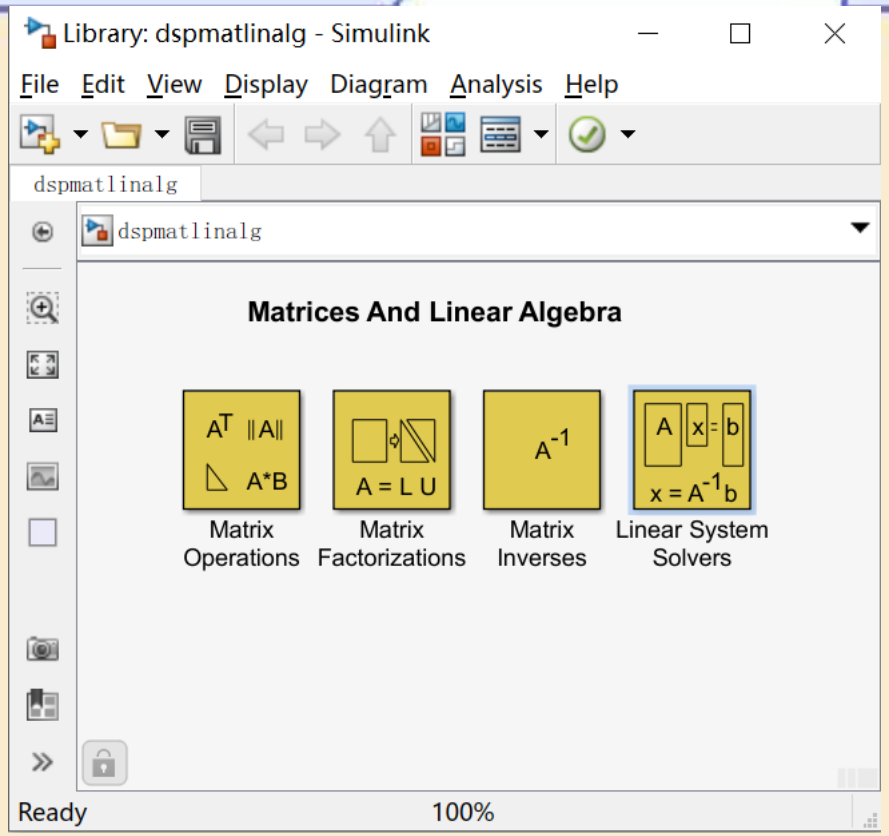
- **在MATLAB命令窗口中直接输入如下指令**

  >>dsplib 也可以打开一个模块库

- **可与第三方共同开发来扩展模块库**

2.2 模块的编辑

1. 添加模块
2. 选取模块
3. 复制与删除模块
4. 模块外形的调整
5. 模块名的处理



2.3 模块的连接

1. 连接两个模块
2. 模块间连线的调整
3. 连线的分支
4. 标注连线
5. 删除连线

2.4 模块的参数和属性设置

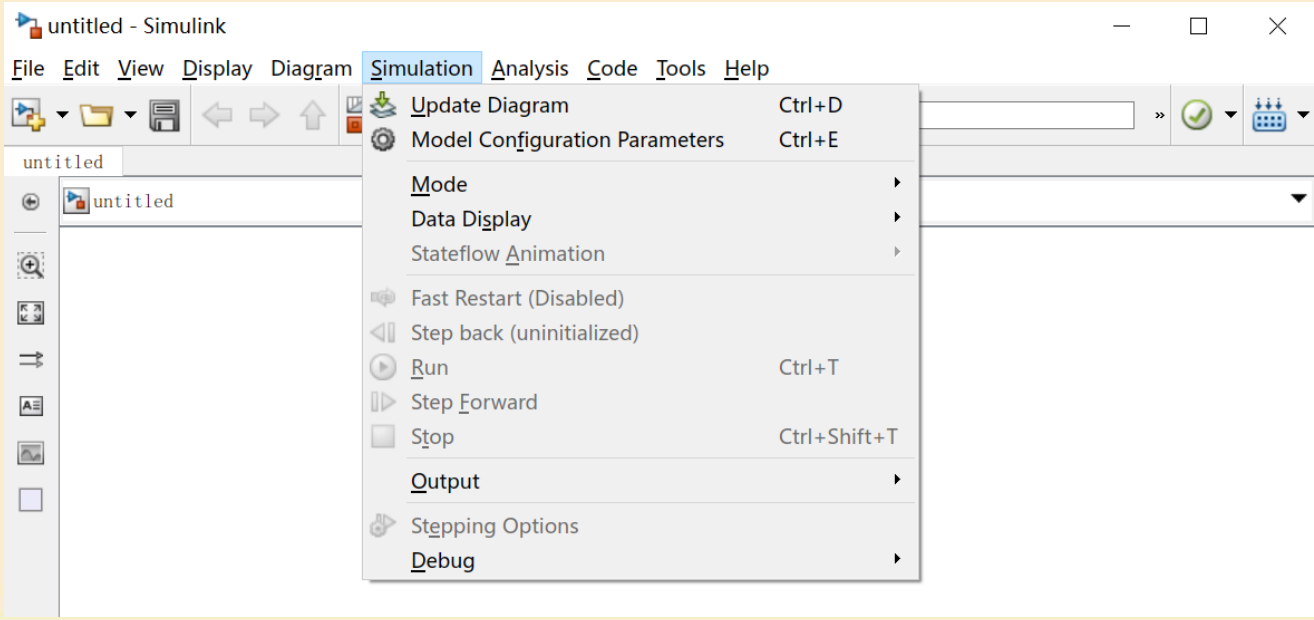
1. 模块的参数设置

Simulink中几乎所有模块的参数都允许用户进行设置，只要双击要设置的模块或在模块上按鼠标右键并在弹出的快捷菜单中选择相应模块的参数设置命令就会弹出模块参数对话框。该对话框分为两部分，上面一部分是模块功能说明，下面一部分用来进行模块参数设置。

同样，先选择要设置的模块，再在模型编辑窗口Edit菜单下选择相应模块的参数设置命令也可以打开模块参数对话框。

2．模块的属性设置

Block properties命令，将打开模块属性对话框。该对话框包括General、Block annotation和Callbacks 3个可以相互切换的选项卡。其中选项卡中可以设置3个基本属性：Description(说明)、Priority(优先级)、Tag(标记)。



11.3 系统的仿真

11.3.1 设置仿真参数

在系统仿真过程中，事先必须对仿真算法、输出模式等各种仿真参数进行设置。其方法是：打开系统仿真模型，从模型编辑窗口的Simulation菜单中选择Configuration Parameters命令，打开一个仿真参数对话框，在其中可以设置仿真参数。在此对话框中，仿真参数设置被分为9个选项。

1．Solver选项设置

在对话框左边的树型结构中单击Solver选项，在右边的窗口中会列出所有Solver选项。

（1）设置仿真起始和停止时间（Simulink time）。

（2）仿真算法的选择（Solver options）。

2．Data Import/Export选项设置

在仿真参数对话框中选择Data Import/Export选项，会出现一个对话框。相应的选项设置分为3组：Load from workspace、Save to workspace和Save options。

（1）从工作空间中载入数据（Load from workspace）。

（2）保存到工作空间（Save to workspace）。

（3）保存选项（Save options）。

3.2 启动系统仿真与仿真结果分析

设置完仿真参数之后，从Simulation中选择Start菜单项或单击模型编辑窗口中的Start Simulation命令按钮，便可启动对当前模型的仿真。此时，Start菜单项变成不可选, 而Stop菜单项变成可选, 以供中途停止仿真使用。从Simulation菜单中选择Stop项停止仿真后，Start项又变成可选。

为了观察仿真结果的变化轨迹可以采用3种方法：

(1) 把输出结果送给Scope模块或者XY Graph模块。

(2) 把仿真结果送到输出端口并作为返回变量，然后使用MATLAB命令画出该变量的变化曲线。

(3) 把输出结果送到To Workspace模块，从而将结果直接存入工作空间，然后用MATLAB命令画出该变量的变化曲线。

例1 试打开一个系统仿真模型示例。

操作过程如下：

(1) 在Simulink主页中，选择New-Simulink Model命令，打开一个模型编辑窗口。

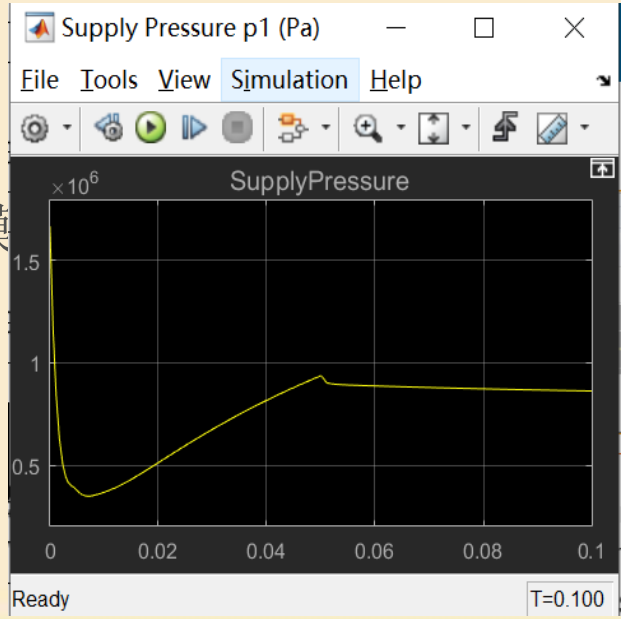
(2) 选择Example选项卡，选择Simulink- Featured Example –Four Hydraulic Cylinder Simulation。

(3)单击 Open Example 按钮。

（4）单击“Run”按钮，可以看到仿真结果。如图

（5）双击模型图标中的 Control Valve Command 模

添加Control Valve Command 子系统图标。



例2 有系统的微分方程，试建立系统仿真模型。

操作过程如下：

(1) 在MATLAB主菜单中，选择File菜单中New菜单项的Model命令，打开一个模型编辑窗口。

(2) 将所需模块添加到模型中。

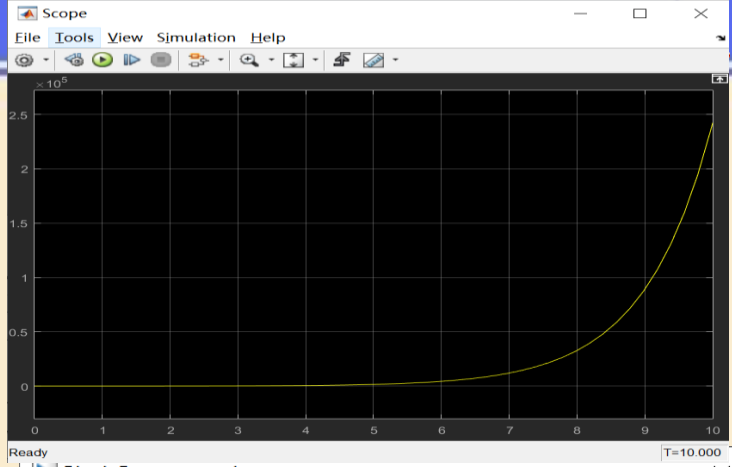
(3) 设置模块参数并连接各个模块组成仿真模型。

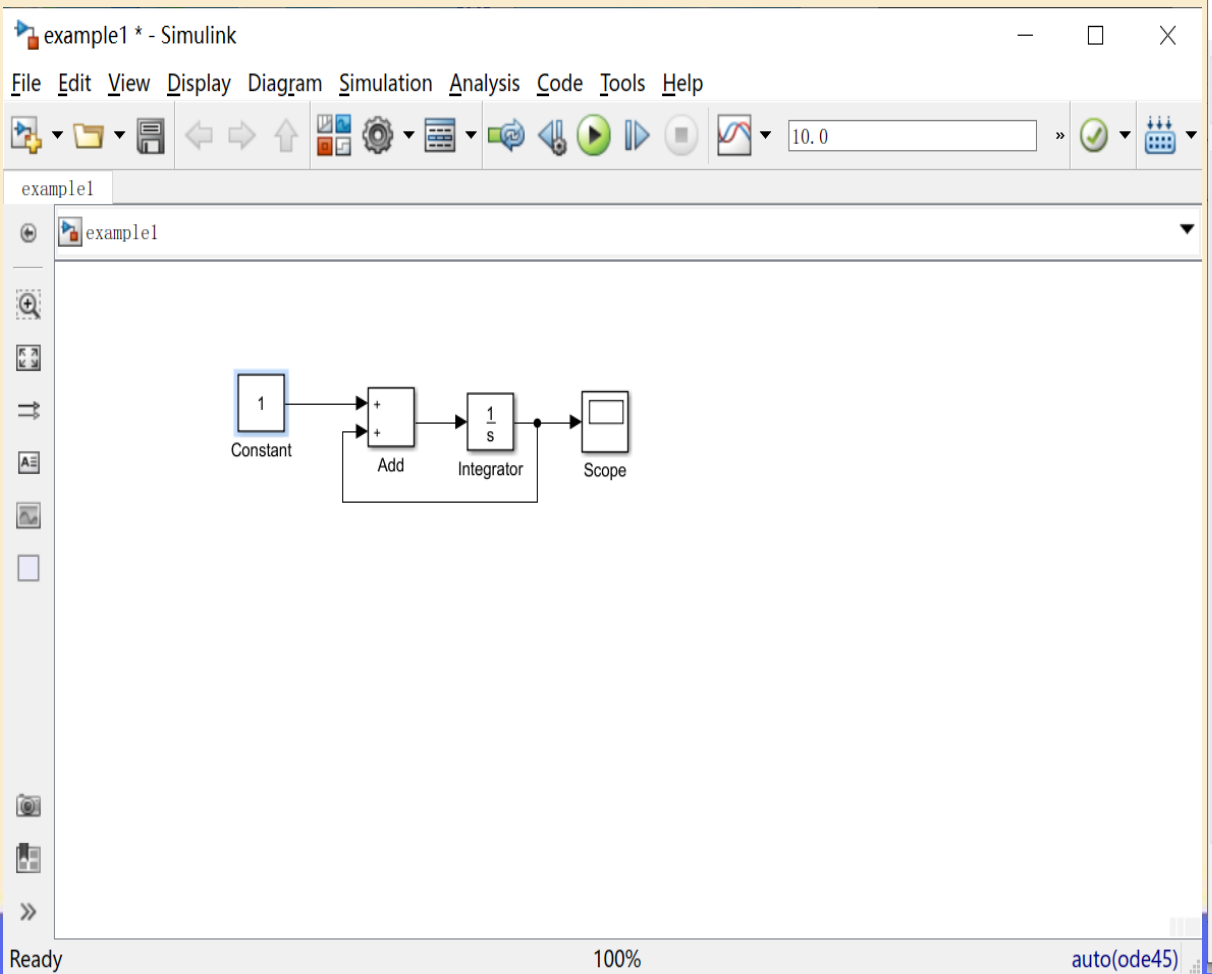
设置模块参数后，用连线将各个模块连接起来组成系统仿真模型。模型建好后，从模型编辑窗口的File菜单中选择Save或Save as命令将它存盘。

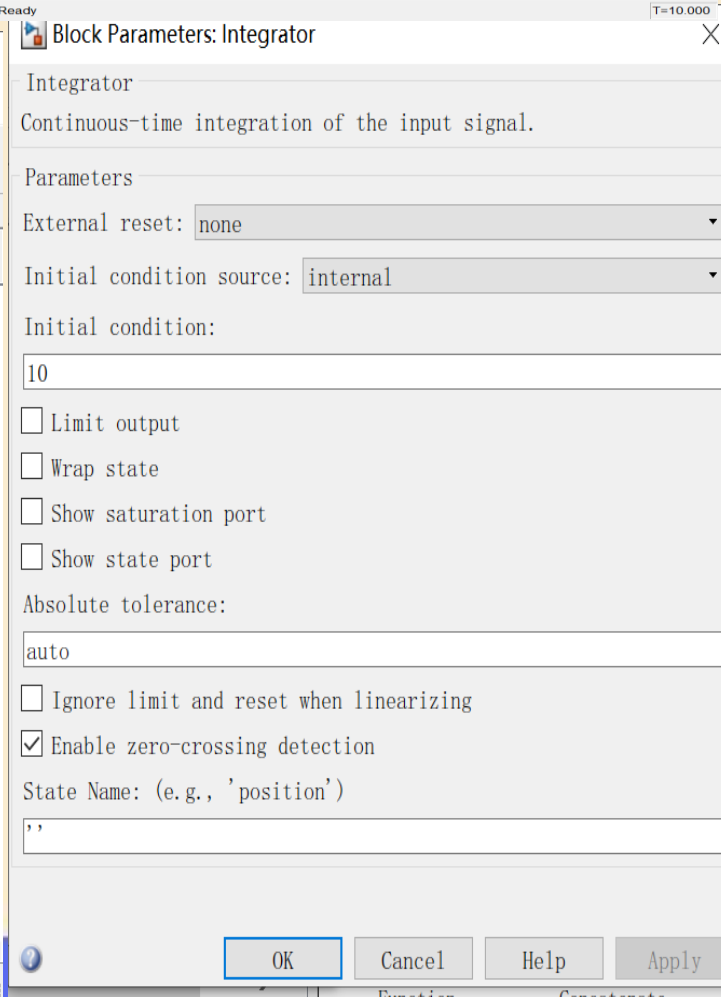
- 使用Simulink求解微分方程

$$\dot{x}=x+1$$

$$x(0)=10$$





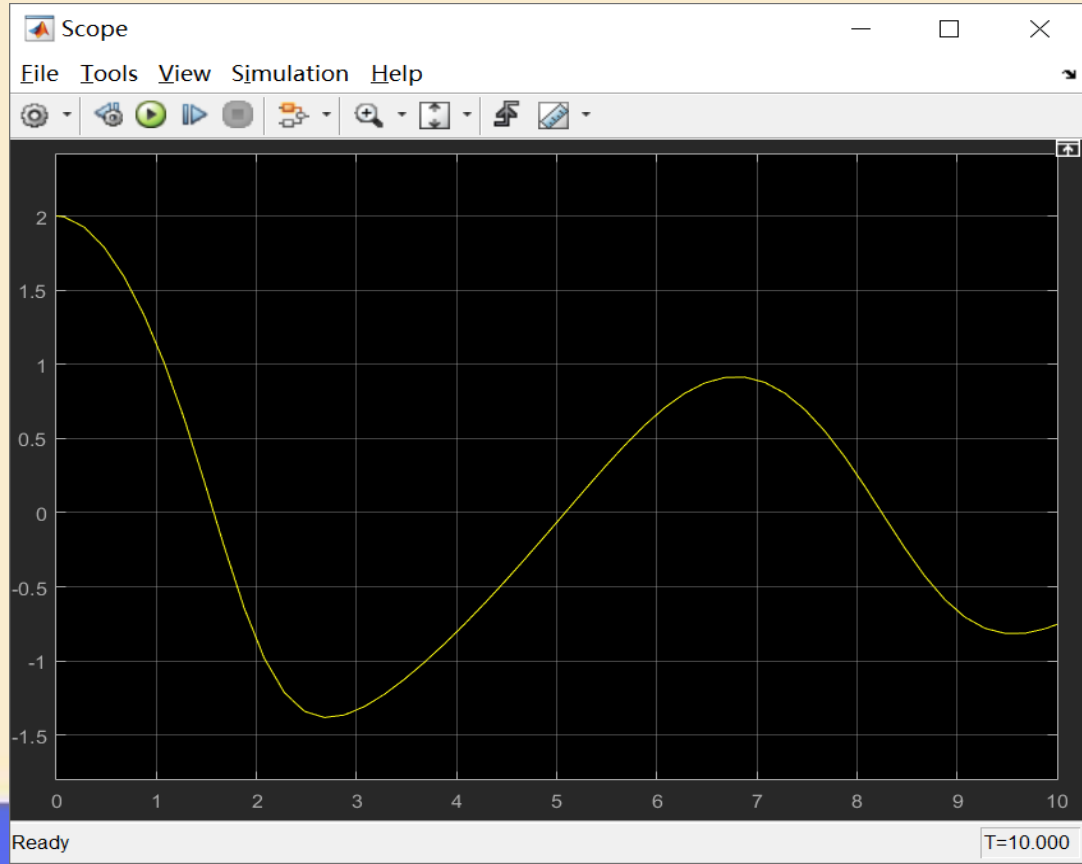


例3 求解2阶微分方程初值问题

$$\ddot{x}=\left(\frac{3}{4}x-\frac{1}{2}x^2\right)\dot{x}-x$$

其中dx(0)=0,x(0)=2。

见example2.slx





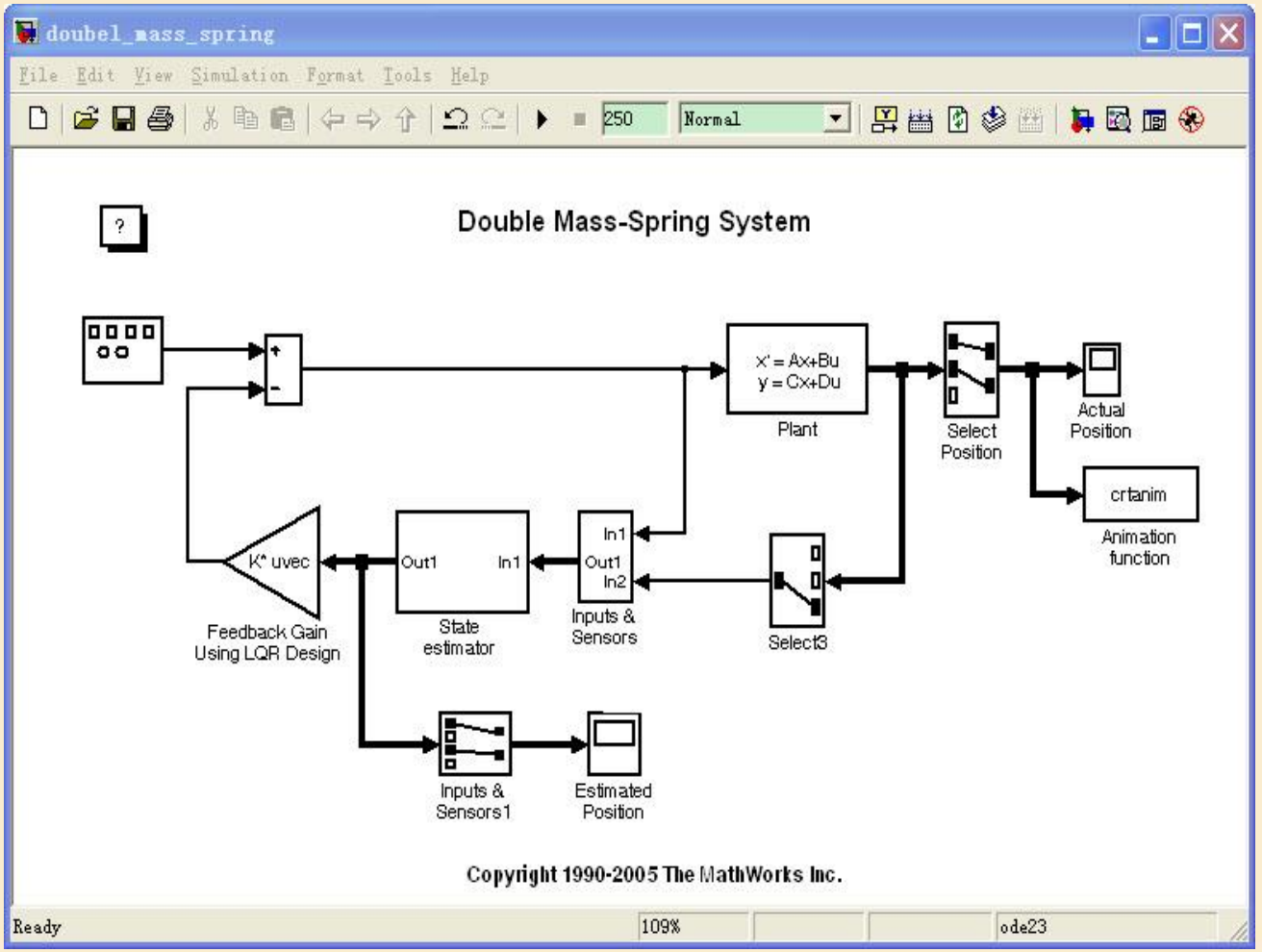
第二节 Simulink动态系统建模与仿真

一、动态系统建模

下面以一个双弹簧质量阻尼系统为例，简要介绍系统的建模过程 。

整个建模过程分为以下几步：

1. 模块选择
2. 模块基本操作
3. 模块连接
4. 模块参数设置



1. 模块选择

➢ 首先新建或者打开一个Simulink文档，即.mdl文件

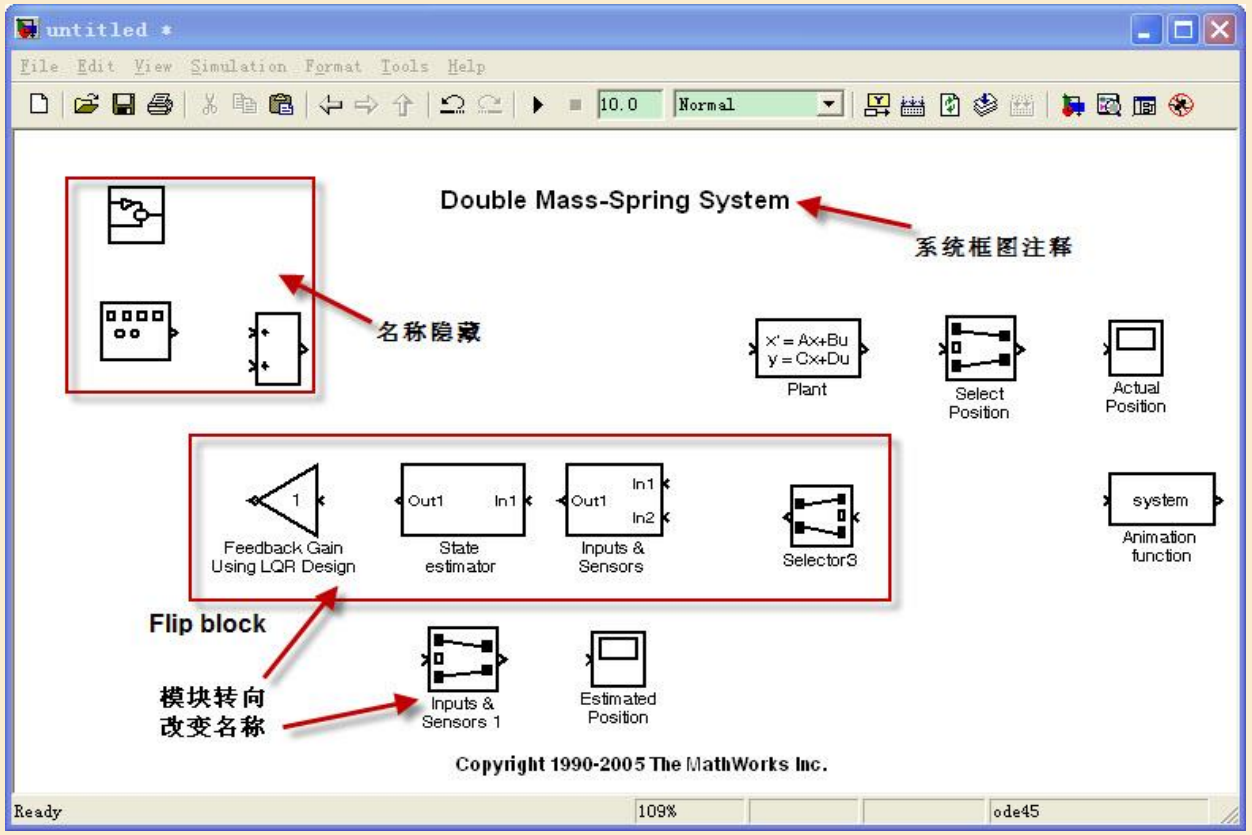
然后在Simulink公共模块库中找到以下模块：

- Subsystem：在Simulink/Ports & Subsystems库中
- Signal Generator：在Simulink/Sources库中
- Add：在Simulink/Math Operations库中
- Gain：在Simulink/Math Operations库中
- Scope：在Simulink/Sinks库中
- In1：在Simulink/Sources库中
- Out1：在Simulink/Sinks库中
- State Space：在Simulink/Continuous库中
- S-Function：在Simulink/User-Defined Functions库中
- Mux：在Simulink/Signal Routing库中
- Selector：在Simulink/Signal Routing库中

拖曳到新建或打开的文件系统模型窗口中

2. 模块基本操作

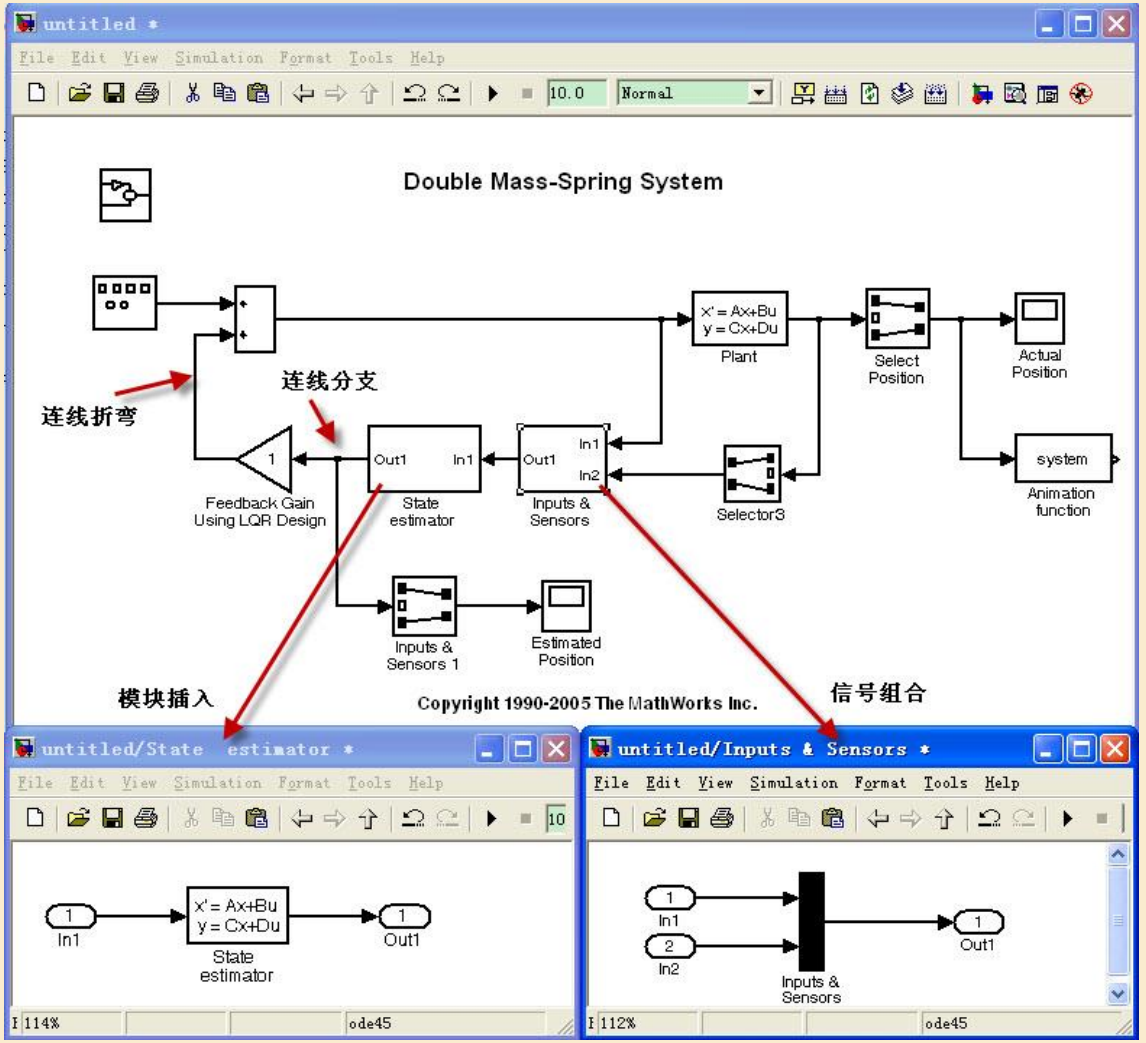
① 移动
② 复制
③ 删除
④ 转向
⑤ 改变大小
⑥ 模块命名
⑦ 颜色设定
⑧ 属性设定
⑨ 输入输出信号



进行上述基本模块操作后，得到右图较为规整的布局图，对其进行连接后，就可完成基本模型的创建

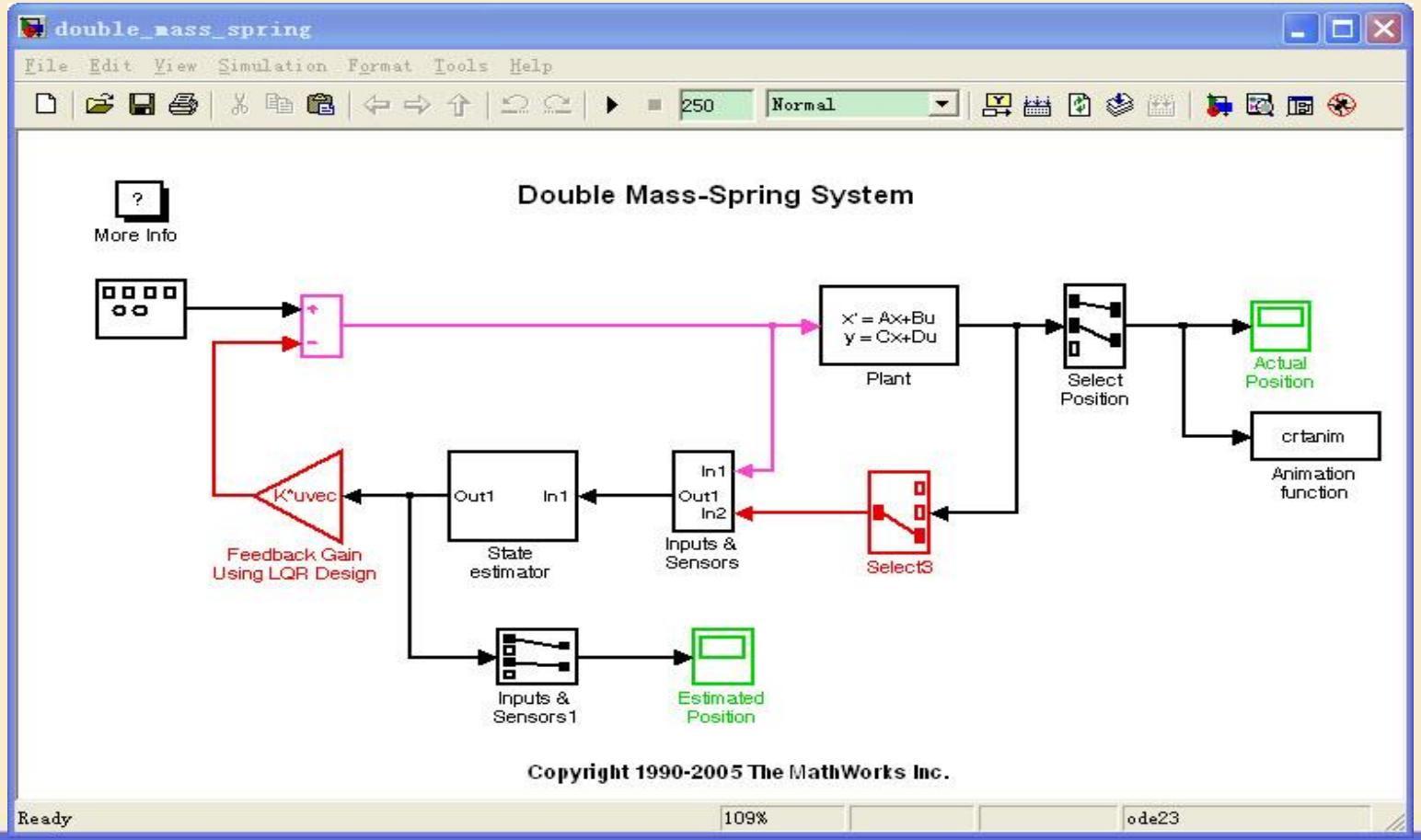
3. 模块连接

用鼠标可以在功能模块的输入端与输出端之间直接连线，按照系统的信号流程通过线将各功能模块连接起来。同时，还可以根据个人喜好，进行一些连线、标签和信号的设置，力求使模型完整、美观、功能齐全。
如右图所示：



4. 模块参数设置

根据系统仿真功能，对各功能模块的参数进行设置，不同功能模块的参数是不同的，但都可以通过用鼠标左键双击功能模块来打开该模块的参数设置窗口，对模块进行参数设定。



设置完毕的模型图

二、动态系统仿真

系统模型建立及模块参数设置后，再对系统仿真参数进行正确的设置，就可以运行仿真了

1. 仿真参数设置

单击模型窗口Simulation菜单下面的Configuration Parameters项或者直接按快捷键Ctrl+E，弹出参数配置界面。单击左侧的Solver选项，进入求解器参数设置面板。设置仿真停止时间为250，求解器为ode23，最大步长为0.1，绝对误差为1e-6，其他为系统默认值。

如果不想出现与求解器相关的警告信息，可以切换到诊断（Diagnostics）面板将Automatic solver parameter selection项设置为‘none’来消除警告信息的显示，不影响系统的仿真。

2. 仿真过程

① 模型编译

Simulink引擎调用模型编译器，模型编译器把模型转换成可执行模式，也就是编译过程。在这个过程中，编译器计算模型中各模块参数表达式的值。确定信号属性，包括名称、数据类型，数值类型和维数，以及模型中未明确说明的信号属性，并检查每个模块能接受的输入信号。属性扩展，确定未声明的属性，包括扩展源信号属性，以及它驱动的模块的输入信号属性。通过用子系统包含的模块替代虚拟子系统来平衡模块层次。确定模块分类顺序，确定模型中没有明确声明采样时间的所有模块的采样时间。

② 链接阶段

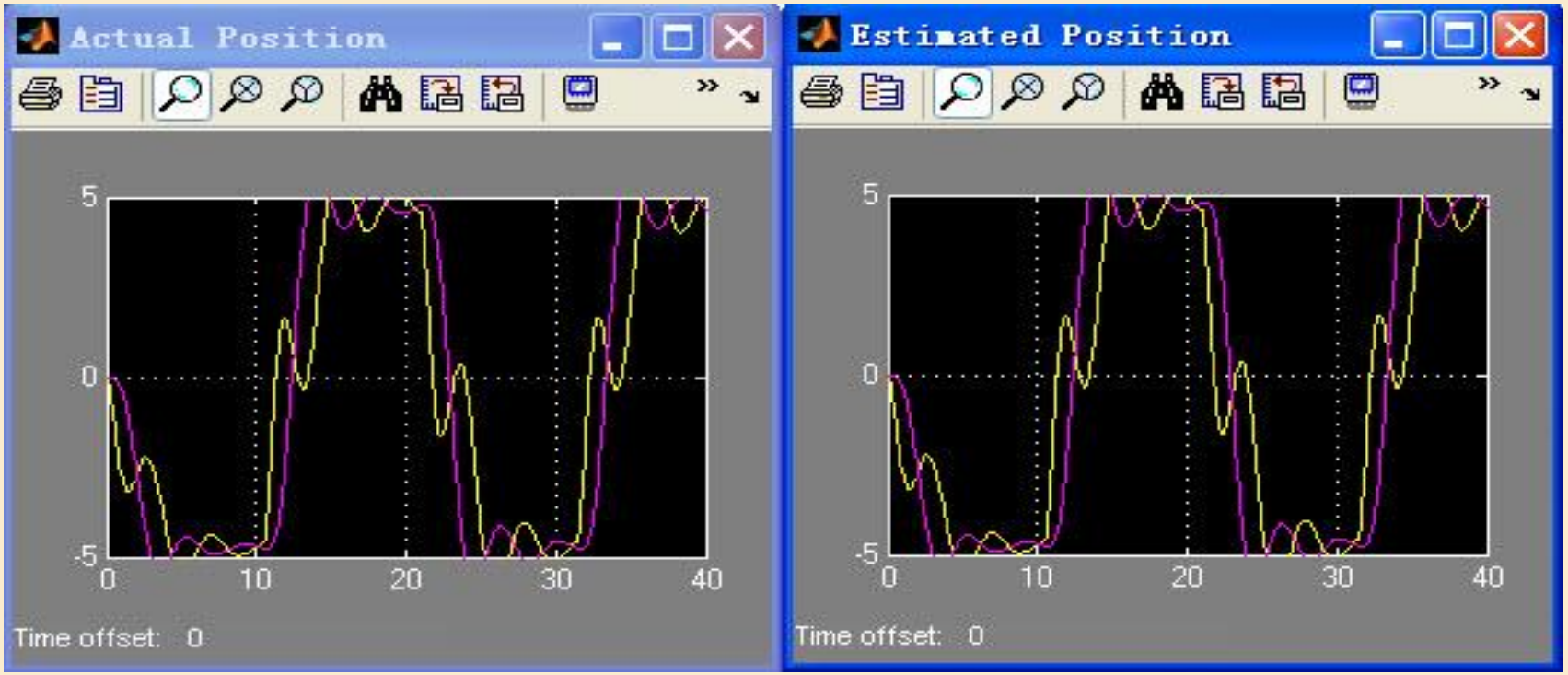
在这个阶段，Simulink引擎为工作区域（包括信号、状态和运行时参数）分配所需的内存，来执行模块表

② 仿真循环阶段

在这个阶段，从仿真开始时间到结束时间的间隔内，Simulink引擎利用模型提供的信息，连续不断地计算系统的状态和输出

3. 仿真结果

仿真结束后，双击两个Scope模块（一个名为Actual Position，另一个名为Estimated Position）会看到系统仿真输出曲线。如下图所示



三、Simulink与MATLAB数据交互

1. 由MATLAB工作空间变量设置系统模块参数

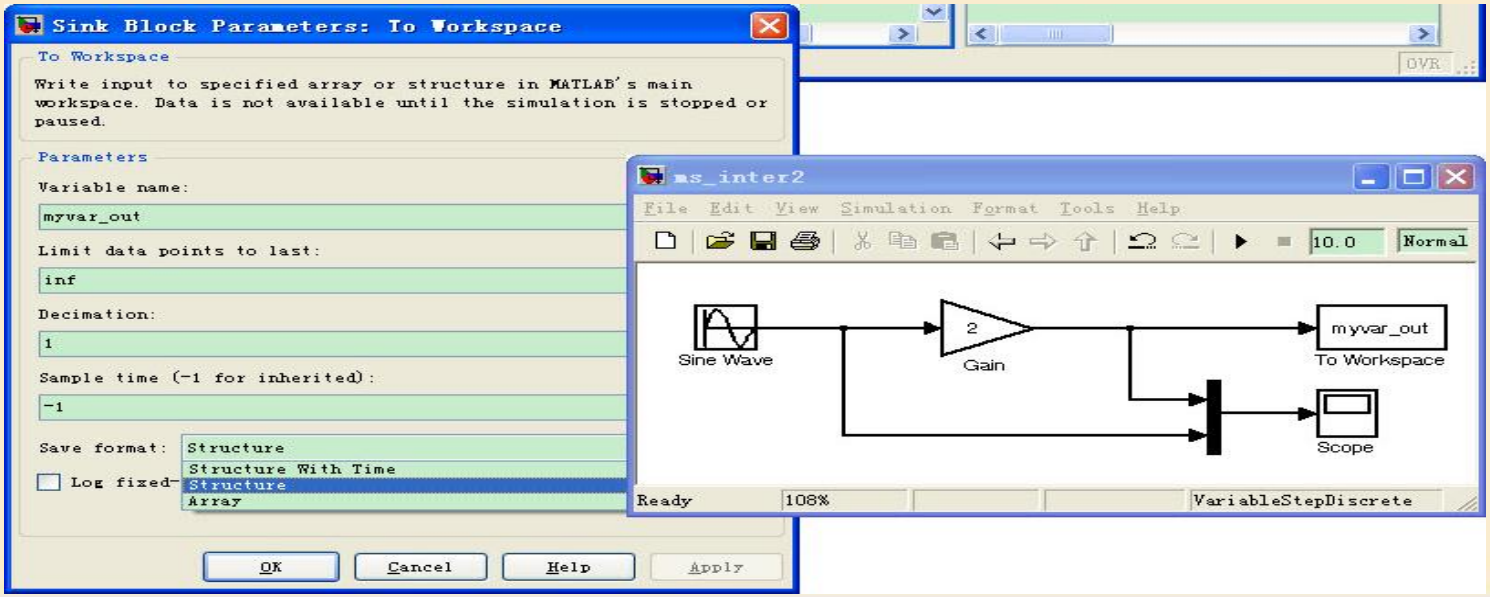
- 用户可以双击一个模块以打开模块参数设置对话框，然后直接输入数据来设置模块参数。其实，用户也可以使用MATLAB工作空间中的变量设置系统模块参数，这对于多个模块的参数均依赖于同一个变量时非常有用。

由MATLAB工作空间中的变量设置模块参数有以下两种形式：

- 直接使用MATLAB工作空间中的变量设置模块参数
- 使用变量的表达式设置模块参数

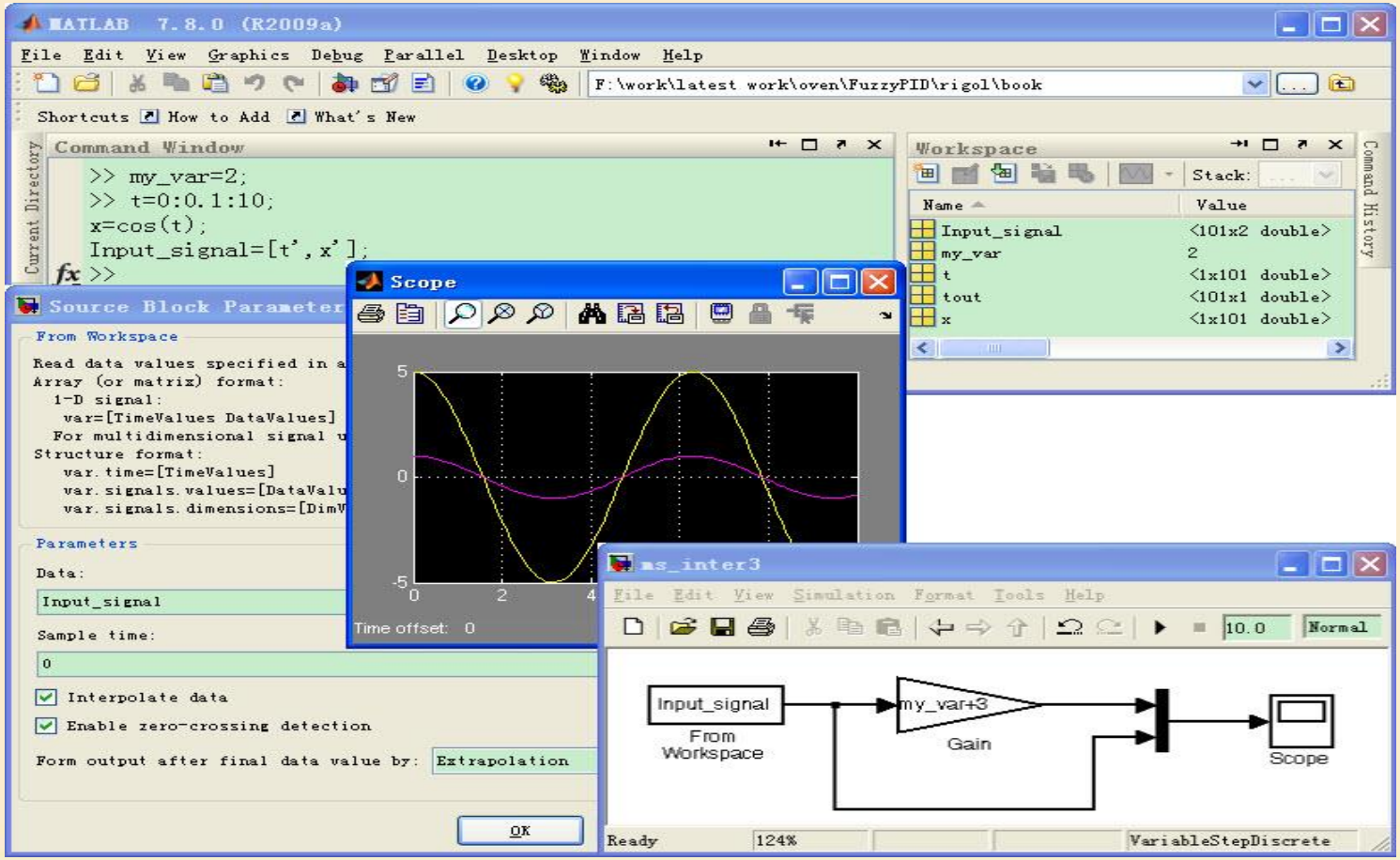
2. 信号输出到工作空间

- 使用Simuilink/Sinks模块库中的To Workspace模块，可以轻易地将信号输出到MATLAB工作空间中。
- 信号输出的名称在To Workspace模块的对话框中设置，此对话框还可以设置输出数据的点数、输出的间隔，以及输出数据的类型等。
- 输出类型有三种形式：数组、结构以及带有时间变量的结构。



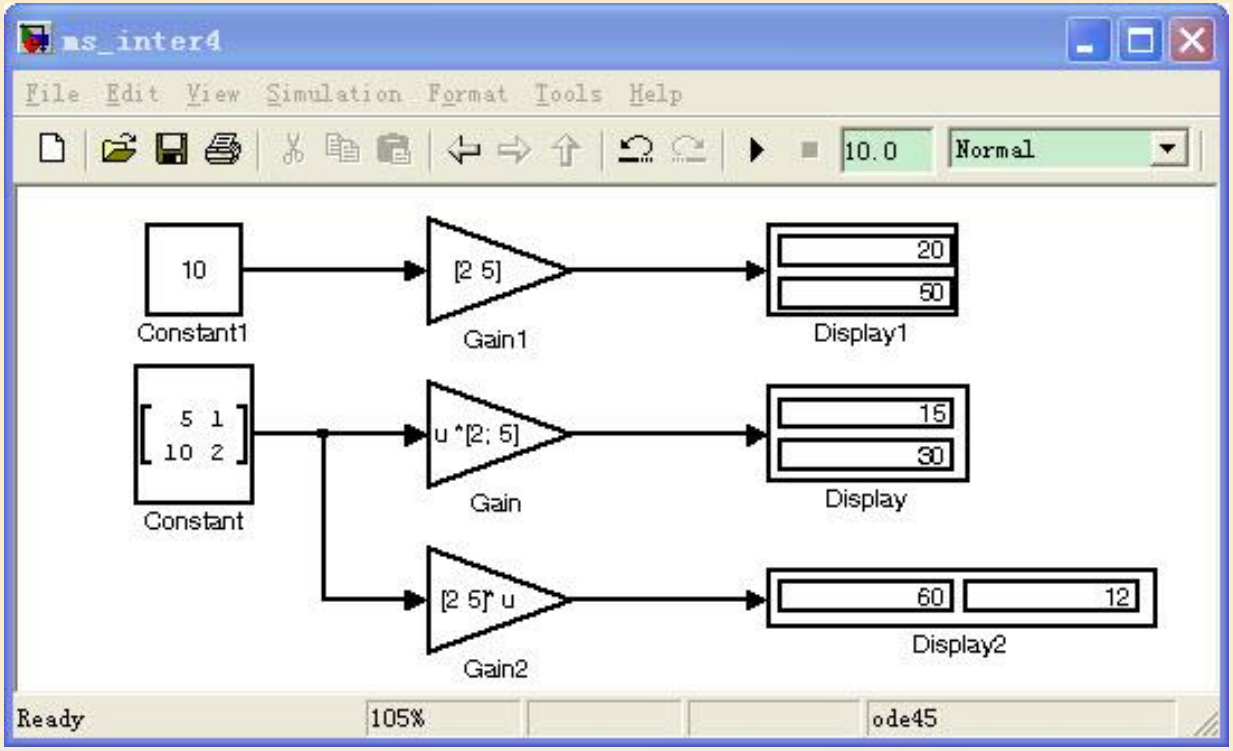
3. 工作空间变量作为系统输入信号

- 使用Simulink/Sources模块库中的From Workspace模块可以将MATLAB工作空间中的变量作为系统模型的输入信号



4. 向量和矩阵形式的信号

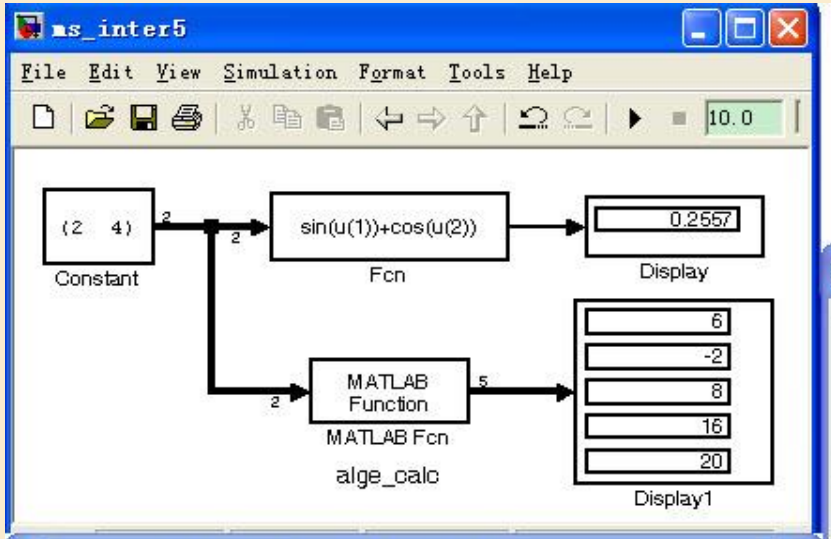
- 在系统模型中，Simulink所使用的信号既可以是标量，也可以传递和使用向量信号。更是支持矩阵形式的信号，它可以区分行和列向量并传递矩阵。通过对模块做适当的配置，可以使模块能够接受矩阵作为模块参数

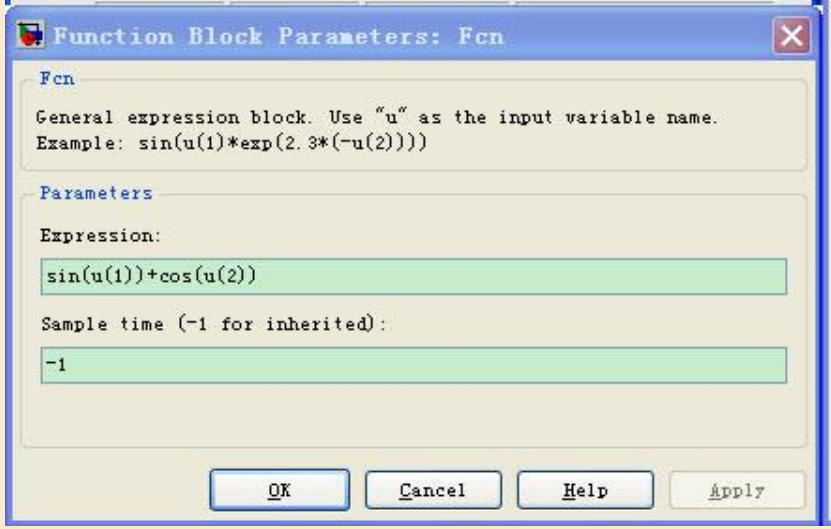


5. MATLAB Fcn 与 Fcn 模块

- 除了上述四种交互方式之外，Simulink与MATLAB之间还可以通过使用Simulink/User-Defined Functions模块库中的MATLAB Fcn模块或Simulink/User-Defined Functions模块库中的Fcn模块进行彼此间的数据交互。
- MATLAB Fcn模块一般用来调用MATLAB函数来实现一定的功能。其调用的函数只能有一个输出（但可以是一个向量）。单输入函数只需要使用函数名，多输入函数输入需要引用相应的元素。
- Fcn模块一般用来实现简单的函数关系，其输入总是表示成u，可以使用C语言表达式。其输出永远为一个标量。

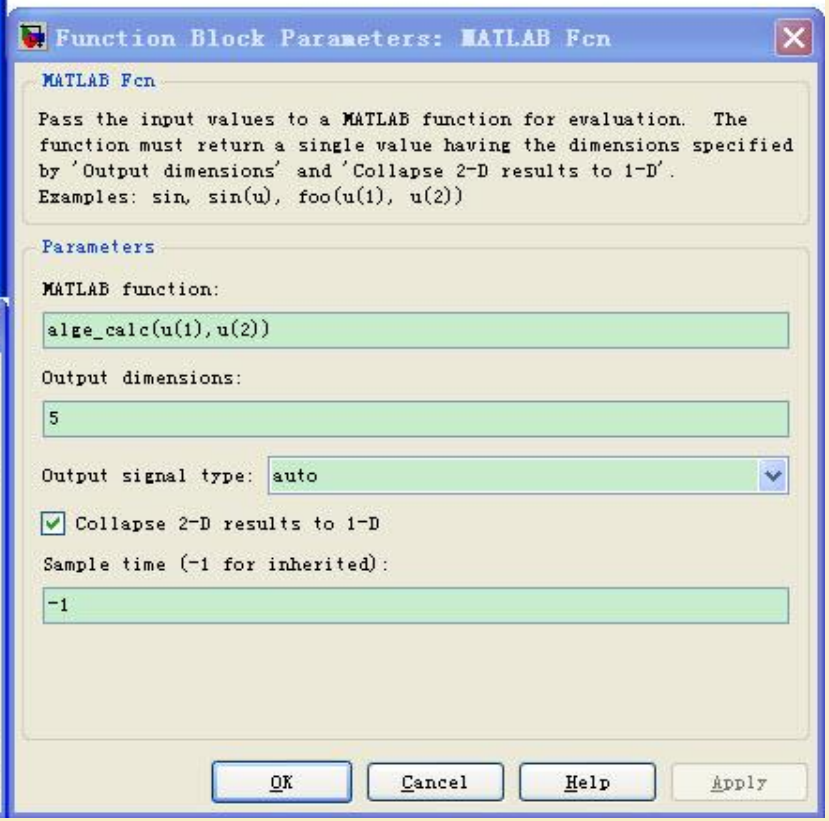
```
function y=alge_calc(a,b)
y1=a+b;  y2=a-b;  y3=a.*b;  y4=2*a+3*b;
y5=a.^2+b.^2;  y=[y1;y2;y3;y4;y5];
```





MATLAB Fcn 调用上述函数

Fcn 直接用表达式



主要内容

- Simulink简介
- Simulink动态系统建模与仿真
- S-Funcition
- 命令行仿真技术
- 综合实例



第三节　S-Function

一、S-Function简介及原理

1. 何为S-Function

- S-Function是系统函数（System Function）的简称，是指采用非图形化的方式描述的一个功能块。S-Function 是由一种特定的语法构成，用来描述并实现连续、离散系统以及复合系统等动态系统；它能够接收来自Simulink求解器的相关信息，并对求解器发出的命令做出适当的响应。
- S-Function 与求解器的交互作用与Simulink系统模块与求解器的交互作用相类似。
- 用户可以采用MATLAB代码（M语言），C，C++，Fortran或Ada等语言编写S-Function。具体使用哪种语言来编写，取决于用户的需要

➢ 什么情况下使用S-Function

当用户所要实现的任务或者功能不能通过现有模块库中的模块来实现，而且即使组合现有模块仍然不能实现想要的功能；此时，就需要用户通过自己编写S-Function来封装自己的C算法和M语言算法，以非图形化的方式描述出一个自定义功能模块。编写S-Function有一套固定的规则，这是由Simulink的仿真机制决定的。

S-Function作为与其他语言相结合的接口，可以使用这个语言所提供的强大能力。如，MATLAB语言编写的S-Function可以充分利用MATLAB所提供的丰富资源，方便地调用各种工具箱函数和图形函数以扩展图形能力；而使用C语言编写的S-Function（C MEX S-Function）还可以实现对操作系统的访问，提高仿真速度，实现与其它进程的通信和同步等。S-Function既可以与已有的代码相结合进行仿真，也可以开发新的Simulink模块。

在动态系统设计、仿真与分析中，用户可以使用Simulink/User-Defined Function模块库中的S-Function模块来使用S-Function。S-Function模块是一个单输入单输出的系统模块，如果有多个输入与多个输出信号，可以使用Mux模块与Demux模块对信号进行组合与分离操作。

一般而言，使用S-Function的步骤如下：

① 创建S-Function源文件。Simulink为用户提供了很多S-Function模板和例子，用户可以根据自己的需要修改相应的模板或例子即可

② 在动态系统的Simulink模型框图中添加S-Function模块，并进行正确的设置

③ 在Simulink模型框图中按照定义好的功能连接输入输出端口，参与模型仿真

2. S-Function 工作原理

- 在对动态系统建模时，总是能够采用广义的状态空间形式对线性系统或非线性系统进行描述。状态方程描述了状态变量的一阶导数与状态变量、输入量之间的关系。
- 一个Simulink模块是由输入、状态和输出三部分所组成，其数学模型如下图所示。当Simulink引擎在仿真一个模型时，模型中的每一个模块的输入，状态和输出之间存在着一定的数学关系

输出方程：

$$y = f(t, x, u)$$

状态(微分)方程：

$$x' = f_d(t, x, u)$$

更新方程：

$$x_{d_{k+1}} = f_u(t, x_c, x_{d_k}, u)$$

式中

$$x = [x_c; x_d]$$

- S-Function是Simulink的重要组成部分，同样也是一个Simulink模块，所以它的仿真原理和仿真过程与前面介绍的Simulink仿真原理是一样的，都体现了状态空间所描述的状态不断更新的特性，同样包括初始化阶段和运行阶段。
- 每个S-Function均由一组S-函数回调方法（S-Function callback methods）组成，在模型仿真期间，在每个特定的仿真阶段，Simulink引擎调用模型中每个S-Function模块相应的回调方法，来完成各个阶段的任务。

这些任务包括：

① 初始化
② 计算下一个采样时间点
③ 计算输出
④ 更新状态
⑤ 数值积分
⑥ 仿真结束

➢ S-Function回调方法及其子函数说明

仿真阶段	S-Function回调方法	子函数	函数作用
初始化阶段	初始化	**mdlInitializeSizes**	定义S-Function模块的基本特性，包括采样时间、连续或离散状态的初始条件和sizes数组
运行阶段	计算下一个采样点	**mdlGetTimeofNextVarHit**	计算下一个采样点的绝对时间，即在mdlInitializeSizes里说明了一个可变的离散采样时间
	计算输出	**mdlOutputs**	计算S-Function的输出
	更新离散状态	**mdlUpdate**	更新离散状态、采样时间和主时间步的要求
	计算导数	**mdlDerivatives**	计算连续状态变量的微分方程
	结束仿真	**mdlTerminate**	实现仿真任务的结束

二、S-Function实现方式及其特点

1. S-Function实现方式

S-Function模块同其他Simulink模块一样，也有直接馈通，同样支持动态可变维数的输入，实现起来要注意到这些功能和特点，避免出错。

常用的几种S-Function实现方式

- Level-1 M-文件S-Function

  只能与S-Function API的一小部分进行交互

- Level-2 M-文件S-Function

  能够与更多扩展的S-Function API进行交互并支持代码生成

- C MEX S-Function

  用户可以用这种方式实现自己的算法或者写一个封装好的S-Function去调用现有的C，C++或者Fortan代码

- **S-Function Builder**

  一种采用图形用户接口来对S-Function功能的子功能进行编程的方式

- **Legacy Code 工具**

  这是帮助用户为合并原有的C或C++代码而创建S-Function的一组MATLAB命令集。象S-Function Builder一样，Legacy Code 工具在代码生成阶段，也能产生将S-Function内联的TLC文件。但这种方式与S-Function Builder和C MEX S-Function相比，只能访问少量的S-Function API 方法。

2. S-Function的特点

- 五种S-Function实现方式，前三种属于需要用户自己来书写代码的形式，而后两种则属于自动生成代码的形式。同时，后三种实现方式又都是C MEX S-Function形式的S-Function

S-Function手工实现方式的特点

实现方式 特 点	Level-1 M-文件S-Function	Level-2 M-文件S-Function	C MEX S-Function
数据类型	支持double类型	支持所有数据类型（包括定点）	支持所有数据类型（包括定点）
数值类型	只支持实数信号	支持实数和复数信号	支持实数和复数信号
帧支持	不支持基于帧的信号	支持基于帧和非帧的信号	支持基于帧和非帧的信号
端口维数	支持向量输入和输出，不支持多路输入和输出	支持标量，一维和多维输入和输出信号	支持标量，一维和多维输入和输出信号
S-函数 API	只支持mdlInitializeSizes mdlDerivatives, mdlUpdate mdlOutputs, mdlTerminate mdlGetTimeOfNextVarHit	除Level 1 M-文件S-Function支持的几个函数外，还支持大量的S-Funciton API	支持全部S-Funciton API
代码生成	不支持	支持，但需要手工书写TLC文件	支持，但需要手工书写TLC文件
Simulink加速模式	运行时解析，因此不能加速	加速模式下提供使用TLC代替运行时解析	加速模式下提供使用TLC或MEX文件选项
模型参考	不能用于参考模型	用于参考模型时，支持标准和加速仿真模式，加速模式需要TLC文件	用于参考模型时，提供采样时间继承选项和标准模式
Simulink.AliasType Simulink.NumericType Simulink.StructType	不支持这些类	支持Simulink.NumericType和Simulink.AliasType	支持所有这些类，但只支持Simulink.StructType作为S-Function参数
总线输入和输出信号	不支持总线输入或输出信号	不支持总线输入或输出信号	不支持总线输入或输出信号
参数调整及运行时参数	支持参数可调，但不支持运行时参数	支持可调参数和运行时参数	支持可调参数和运行时参数
工作向量	不支持工作向量	支持D工作向量	支持所有工作向量类型

S-Function自动实现方式的特点

实现方式 特 点	S-Function Builder	Legacy Code Tool
数据类型	支持所有数据类型（包括定点）	支持所有内联数据类型，使用定点类型时，必须声明数据类型为Simulink.NumericType。没有进行定标的定点数据不能使用
数值类型	支持实数和复数信号	只支持内联数据类型的复数信号
帧支持	支持基于帧和非帧的信号	不支持基于帧的信号
端口维数	支持标量，一维和多维输入和输出信号	支持标量，一维和多维输入和输出信号
S-函数 API	支持mdlInitializeSizes, mdlInitializeSampleTimes. mdlDerivatives, mdlUpdate mdlOutputs, mdlTerminate mdlStart	支持mdlInitializeSizes, mdlInitializeSampleTimes. mdlStart，mdlInitializeConditions, mdlOutputs, and mdlTerminate
代码生成	支持，且自动生成TLC文件	支持嵌入式系统的代码生成优化，也自动生成支持内联S-Function 表达式叠起的TLC文件
Simulink加速模式	如果生成了TLC文件，则在加速模式下使用TLC文件，如果没有生成，则使用MEX-文件	加速模式下提供使用TLC或MEX文件选项
模型参考	用于参考模型时，使用缺省值	用于参考模型时，使用缺省值
Simulink.AliasType Simulink.NumericType Simulink.StructType	不支持这些类	支持Simulink.AliasType和Simulink.NumricType
总线输入和输出信号	不支持总线输入或输出信号	支持总线输入和输出信号，但要先在MATLAB工作空间定义一个Simulink.Bus对象，等价于在使用Legacy Code的输入或输出结构。不支持总线参数
参数调整及运行时参数	只在仿真期间支持可调参数。 支持运行时参数	支持可调参数和运行时参数
工作向量	不提供访问工作向量	在使用SS_DWORK_USED_AS_DWORK时，支持D工作向量

3. S-Function实现方式选择

- 用户应尽量采用Simulink和各种模块库中的模块来搭建系统模型，除非模块库中的模块不能满足要求，或者用户确实关注仿真速度，才宜编写S-Funciton，以免带来不必要的错误。
- 就编码型的M语言S-Function和C MEX S-Function来说，各有优缺点。C MEX S-Function不仅能够用于Simulink仿真，也可以用于通过Real-Time Workshop转成面向不同目标代码，而M语言S-Function只能用于Simulink仿真（只限于level 1 M-文件）。用C MEX S-Function比用M语言S-Function复杂，但一般来讲运行速度更快一些，如果速度是用户关注的焦点，则宜用C MEX S-Function。

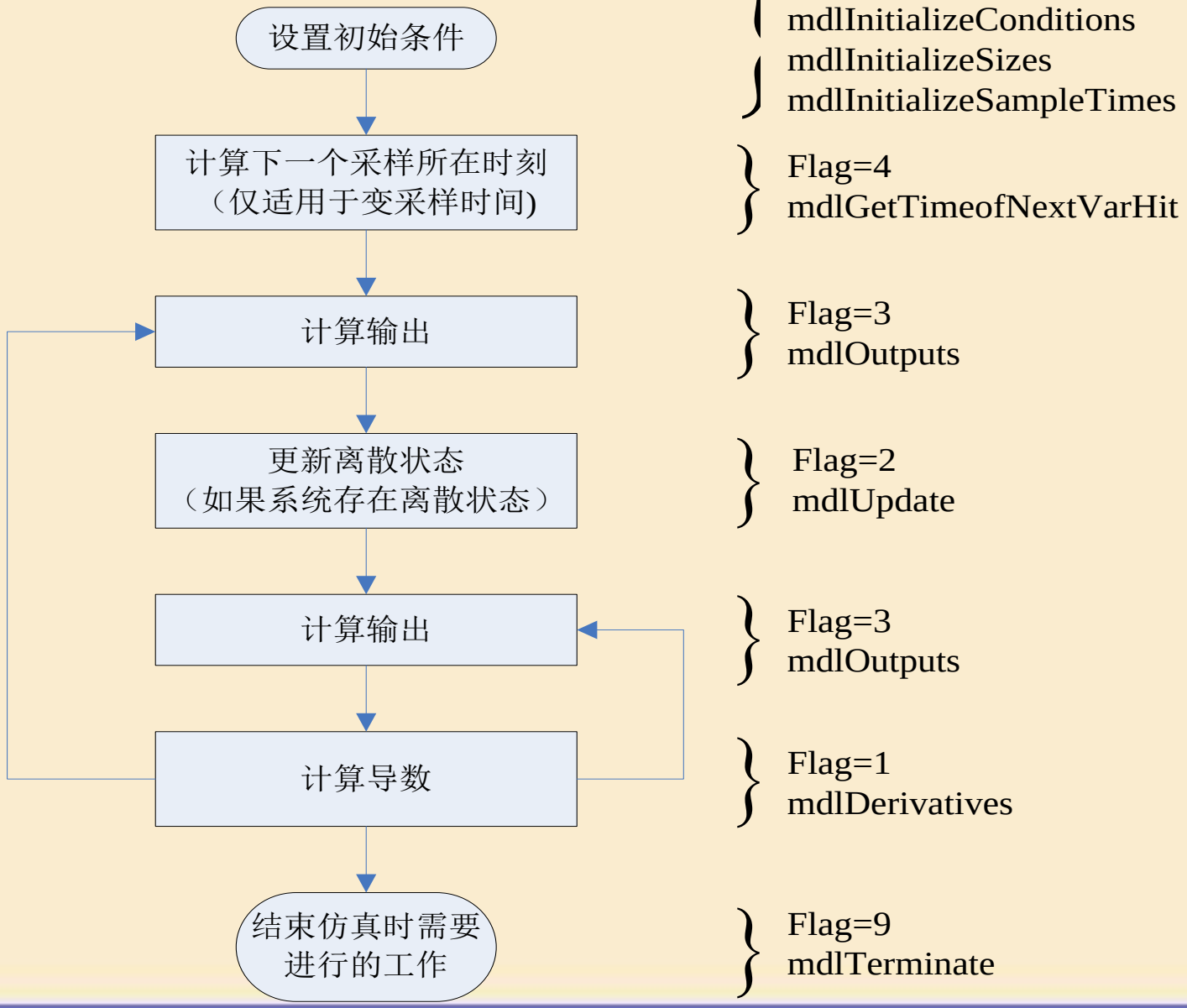
三、M 语言 S-Function

1. Level-1 M文件 S-Function

Simulink为手工编写S-Function提供了各种模板文件，其中定义了S-Function完整的框架结构。用户可以根据自身的需要，对模板文件加以剪裁，形成自己的S-Function。

Level-1 M-文件S-Function的模板文件为sfuntmpl.m，这个文件包含了一个完整的Level-1 M-文件S-Function，它包含一个主函数和6个子函数。在主函数内程序根据标志变量flag，由一个Switch语句结构根据不同的flag值，执行流程转移到相应的子函数。flag标志作为主函数的参数由系统（Simulink引擎）调用时给出。

工作流程如图所示：


设置初始条件
计算下一个采样所在时刻
（仅适用于变采样时间)
计算输出
更新离散状态
（如果系统存在离散状态）
计算输出
计算导数
结束仿真时需要
进行的工作
Flag=0
mdlInitializeConditions
mdlInitializeSizes
mdlInitializeSampleTimes
Flag=4
mdlGetTimeofNextVarHit
Flag=3
mdlOutputs
Flag=2
mdlUpdate
Flag=3
mdlOutputs
Flag=1
mdlDerivatives
Flag=9
mdlTerminate

一个Level-1 M-文件S-Function的基本形式如下：

[sys,x0,str,ts,simStateCompliance] = my_sfunc(t,x,u,flag,p1,p2,...)

my_sfunc是S-Function的函数名，用户可以根据自己函数的功能，取易于理解又能表达函数功能的由MATLAB支持的字符构成的函数名。在模型仿真期间，Simulink引擎根据flag标志的不同，不断重复调用my_sfunc来完成不同阶段的任务，并把结果通过输出向量返回

仿真时，Simulink引擎传递以下参数给Level-1 M-文件S-Function。各输入参数说明下表

S-Function主函数输入参数及其说明

输入参数	说　明
t	当前仿真时间。通常用于决定下一个采样时刻，或者在多采样速率系统中，用来区分不同的采样时刻点，并据此进行不同处理
x	状态向量。即使在系统中不存在状态时，这个参数也是必需的
u	输入向量
flag	标识符。控制在每个仿真阶段调用哪一个子函数，由Simulink在调用时自动取值
p1,p2…pn	可选参数。这是由用户根据功能需要，自动添加提供给S-Function的，可用于任何一个子函数中

Level-1 M-文件S-Function返回的输出向量包含以下参数

S-Function主函数输出参数及其说明

输出参数	说　明
sys	通用的返回参数，其返回值的意义取决于flag的值
x0	初始状态值
str	保留值，必须设为空矩阵
ts	采样周期变量，两列分别表示采样时间间隔和偏移，即[Tperiod, Toffset]
simStateCompliance	当保存或恢复模型全部仿真状态时，说明如何处理这个S-函数模块。其取值在模板文件中给出了详细说明。这一项是为了向level 2 M 文件S-函数转化在MATLAB 2009a版本中新增加的一个输出参数，以前的版本中没有这个参数

2. Level-2 M文件 S-Function

- Level-2 M-文件S-Function中引入了面向对象技术，涉及到类的知识。它本身就是一个M文件，文件中定义了一个实例的属性和行为，这是在Simulink模型中一个引用M-文件的Level-2 M-文件S-Function模块的实例。M-文件本身包含一组回调方法，当更新或仿真模型时，供Simulink引擎调用。这些回调方法完成由S-Function定义的模块的初始化和计算模块的输出
- Simulink引擎将一个运行时对象传递给回调方法作为参数。这个运行时对象作为S-Function模块的M代理，有效的进行服务，允许回调方法在仿真或模型更新期间设置和访问模块的属性

(1) 运行时对象（Run-Time Object）

- 当Simulink引擎调用Level-2 M-文件S-Function回调方法时，它传递一个Simulink.MSFcnRunTimeBlock 类的实例给回调方法作为参数。 这个作为S-Function模块运行时对象的实例，与服务于C MEX S-Function回调方法的SimStruct结构目的相同。这个对象使能回调方法提供和获得关于模块端口各种元素、参数、状态和工作向量的信息。类的方法则通过取得或设置属性或者调用模块运行时对象的方法来完成这项工作。取得或设置运行时对象属性和调用运行时对象方法的内容可以参考Simulink.MSFcnRunTimeBlock 类的说明文档。
- 运行时对象不支持MATLAB稀疏矩阵。

(2) Level-2 M-文件S-Function回调方法

- Level-2 M-文件S-Function API 定义了构成Level-2 M-文件S-Function的回调方法的签名和一般用途。S-Function本身提供了这些回调方法的实现，这些实现反过来确定模块的属性和行为。通过创建带有一组合适的回调方法的S-Function，用户就能够定义一个满足自己特定应用需要的模块。
- Level-2 M-文件S-Function必须包括以下回调方法：
  - Setup函数来初始化基本的S-Function特征
  - Outputs函数来计算S-Function的输出
- Setup方法在对与Level-2 M-文件S-Function模块相对应的实例进行初始化时，类似于回调方法mdlInitializeSizes和mdlInitializeSampleTimes

一个简单的Level-2 M-文件S-Function基本步骤如下：

- 拷贝Level-2 M-文件S-Function 模板msfuntmpl.m到你的工作文件夹，将模板名更改为能够描述自己函数的文件名，同时将.m文件中第一行的函数名也改为与文件名相同的名字。
- 修改Setup方法来初始化S-Function 属性。
- 在PostPropagationSetup方法中初始化离散状态，Level-2 M-文件S-Function在一个D工作向量中存储离散状态信息。
- 在InitializeConditions或Start回调方法中初始化离散和连续状态值或者其他D工作向量。用Start回调方法初始化那些只需在仿真开始时初始化一次的变量值，用InitializeConditons方法来初始化那些包含S-Function的使能子系统在任何时候再次使能时需要重新初始化的变量值。
- 在Outputs回调方法中计算S-Function的输出。
- 对于具有连续状态的S-Function，在Derivatives回调方法中计算状态微分。运行时对象在他们的Derivatives属性中存储微分数据。
- 在Update回调方法中更新任何离散状态。
- 在Terminate回调方法中完成所需的清除任务，如清除变量或内存。

3. Level-1 S-Function 转换 Level-2 S-Function

- 通过将Level-1 M-文件 S-Function的每一个flag标志内的代码映射到Level-2 M-文件 S-Function相应的回调方法中，用户可以实现Level-1 S-Function到Level-2 S-Function的转换

除此之外，还要做如下工作才能实现真正的转换。

- 在D工作向量中存储Level-2 M-文件S-Function的离散状态信息，在PostPropagationSetup方法中进行初始化
- 使用DialogPrm运行时对象属性代替传递函数参数的形式来访问Level-2 M-文件S-Function的对话框参数
- 对具有变采样时间的S-Function，Level-2 M-文件S-Function需要在Outputs方法中更新NextTimeHit运行时对象属性来设置下一时刻采样时间

四、C MEX S-Function

- 在仿真期间，C MEX S-Function必须向Simulink引擎提供函数信息。仿真过程中，Simulink引擎，ODE求解器和C MEX S-Function交互作用来完成特定的任务。

用C语言编写S-Function具有以下优点：

- 执行速度快
- 实时代码生成
- 包含已有的C代码
- 能够访问操作系统接口
- 可以编写设备驱动

1. C MEX S-Function的几个概念

(1) MEX文件

- 对于M文件S-Function，在MATLAB环境下可以通过解释器直接执行。对于C语言或其它语言编写的S-Function，则需要先编译成可以在MATLAB内运行的二进制代码：动态链接库或者静态库，然后才能使用。这些经过编译的二进制文件就是所谓的MEX文件，在Windows系统下早期版本的MATLAB中MEX文件后缀为dll，现在则为mexw32或mexw64。要将C文件S-Function编译成动态库，需在MATLAB命令行下输入：

```
>> mex my_sfunction.c
```

使用mex命令之前，需要在系统中安装一个C编译器

(2) SimStruct数据结构

一个称为SimStruct的数据结构描述了S-Function中所包含的系统。此结构在头文件simstruc.h中定义，SimStruct将描述系统的所有信息，即封装系统的所有动态信息。它保存了指向系统的输入、状态、时间等存储区的指针，另外它还包含指向不同S-Function回调方法的指针。

实际上整个Simulink框图模型本身也是通过一个SimStruct数据结构来描述的，它可以被视为与Simulink框图模型等价的表达。模型中的每一个S-Function模块也都有自己的SimStruct。这些SimStructs的组成就象一个目录树，与模型相关联的SimStruct是根（root）SimStruct，与S-Function相关联的SimStruct是子SimStruct。

(3) 工作向量（Work Vector）

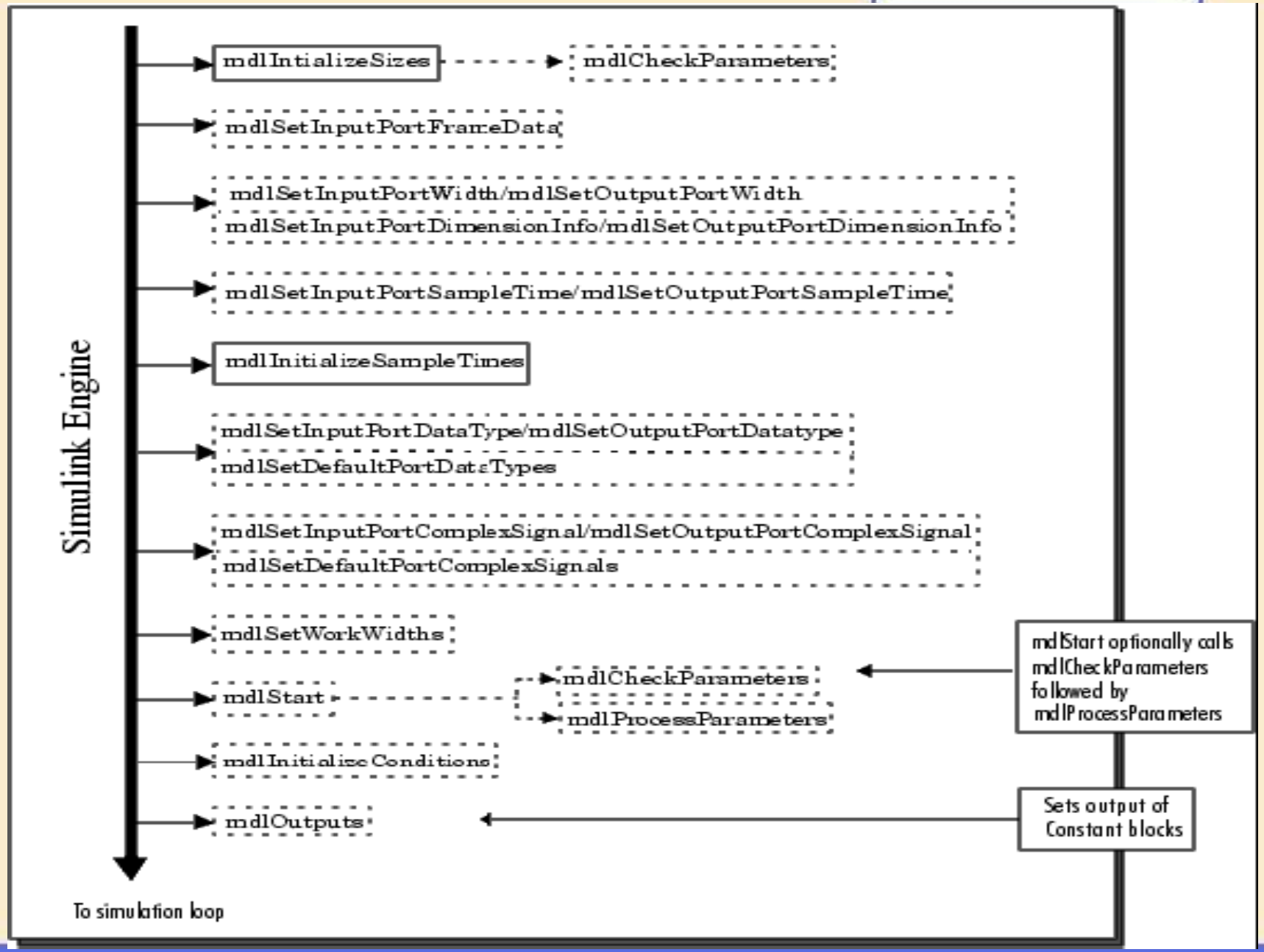
在仿真过程中不释放的内存区称之为持续存储区（Persistent Memory Storage），为全局变量或局部静态变量分配的内存就是这样的区域。当一个模型中出现同一个S-Function的多个实例时，这些全局变量或者局部静态变量就会发生冲突，导致仿真不能正确进行。因为这些实例使用了共同的动态链接库（MEX文件），正如在Windows下多个实例在内存中只有一个映像一样。此时Simulink为用户提供了工作向量来解决这个问题。

工作向量是Simulink为每个S-Function实例分配的持续存储区，它完全可以替代全局变量和局部静态变量。

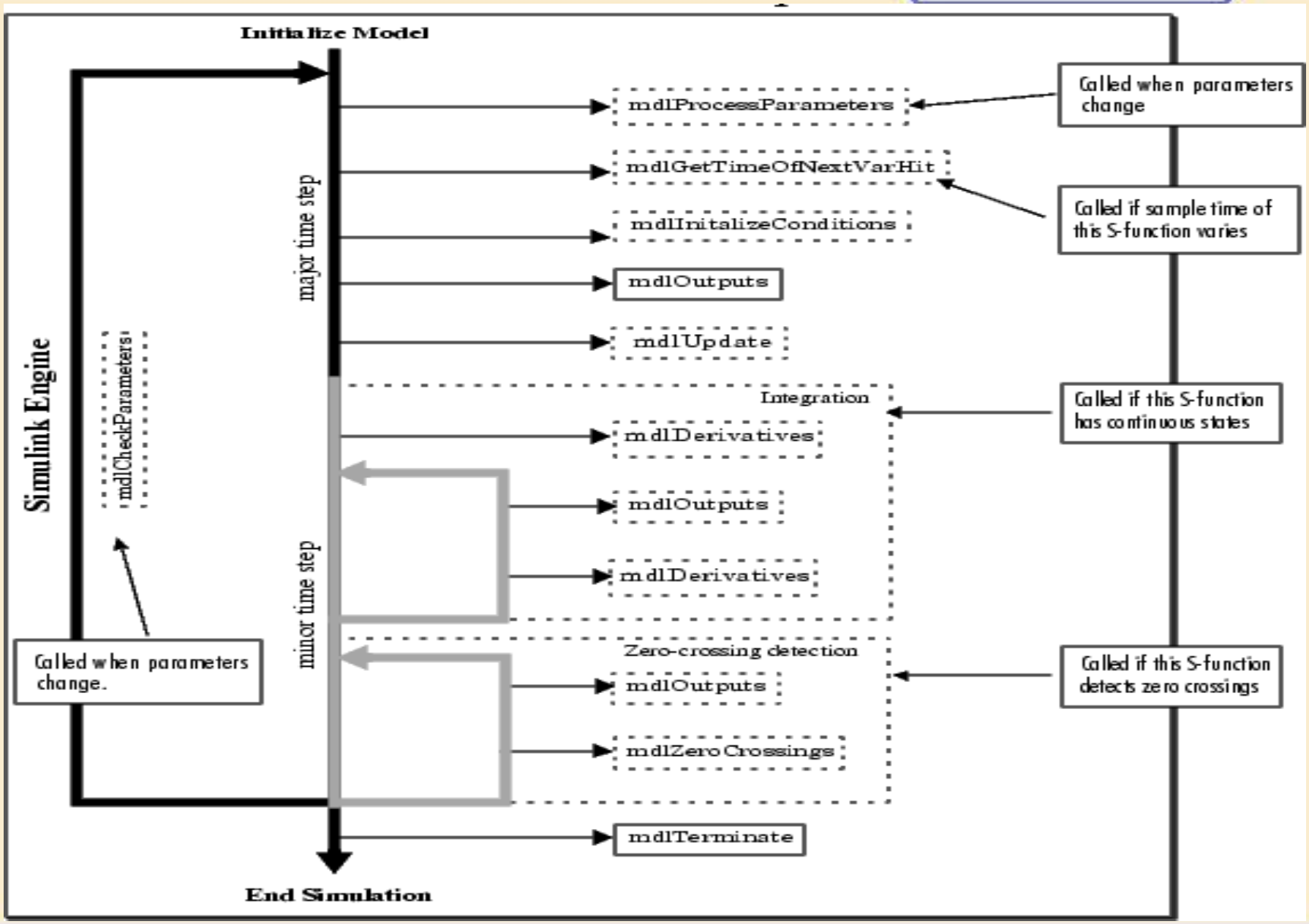
常用的工作向量函数见书中表22.3-9

2. C MEX S-Function 流程

- C MEX S-Function 与 M-文件 S-Function 完成系统仿真的流程相似，不同的是C MEX S-Function的流程控制更为精细，数据I/O也更为丰富。除了常用的输入输出数据外，S-Function还经常用到的内部数据有：连续状态、离散状态、状态导数和工作向量。访问这些数据首先需要一套宏函数来获取指向存储它们的存储器的指针，然后通过指针来访问。此时需要特别注意指针的越界访问问题。
- 下图给出了Simulink引擎在一个S-Function中调用其回调方法的顺序。实矩形框表示在模型初始化期间或在每一时间步，回调总在发生。虚线矩形框表示在仿真循环期间或一些时间步上，回调可能会发生。在上面的模型初始化环中，Simulink引擎为将要开始的仿真配置S-Function。引擎总是需要调用mdlInitializeSizes和mdlInitializeSamppleTime来设置S-Function的基本属性，包括输入和输出端口，S-Function对话框参数，工作向量和采样时间等


Simulink Engine
mdlIntializeSizes
mdlCheckParameters
mdlSetInputPortFrameData
mdlSetInputPortWidth/mdlSetOutputPortWidth
mdlSetInputPortDimensionInfo/mdlSetOutputPortDimensionInfo
mdlSetInputPortSampleTime/mdlSetOutputPortSampleTime
mdlInitializeSampleTimes
mdlSetInputPortDataType/mdlSetOutputPortDatatype
mdlSetDefaultPortDataTypes
mdlSetInputPortComplexSignal/mdlSetOutputPortComplexSignal
mdlSetDefaultPortComplexSignals
mdlSetWorkWidths
mdlStart
mdlCheckParameters
mdlProcessParameters
mdlInitializeConditions
mdlOutputs
mdlStart optionally calls mdlCheckParameters followed by mdlProcessParameters
Sets output of Constant blocks
To simulation loop

- Simulink引擎总是如所需要那样，调用相应的方法来完成S-Function的初始化。例如，如果S-Function使用工作向量，引擎就调用mdlSetWorkWidths方法。再者，如果mdlInitializeSizes方法延缓设置了输入和输出端口属性，Simulink引擎将调用任何其他能用的方法来完成端口的初始化，如在信号传播期间调用mdlSetInputPortWidth。如果S-Function使用对话框参数，mdlStart方法将调用mdlCheckParameters和mdlProcessParameters方法。
- 初始化之后，Simulink引擎执行下图所示的仿真环。如果仿真循环被人为或者发生一个错误所中断，引擎直接跳转到mdlTerminate方法。如果仿真被人为停止，引擎先结束当前时间步，然后再调用mdlTerminate方法。


Initialize Model
Simulink Engine
major time step
minor time step
mdlCheckParameters
mdlProcessParameters
mdlGetTimeOfNextVarHit
mdlInitalizeConditions
mdlOutputs
mdlUpdate
Integration
mdlDerivatives
mdlOutputs
mdlDerivatives
Zero-crossing detection
mdlOutputs
mdlZeroCrossings
mdlTerminate
End Simulation
Called when parameters change
Called if sample time of this S-function varies
Called if this S-function has continuous states
Called if this S-function detects zero crossings
Called when parameters change.

3. C MEX S-Function 模板

- 在MATLAB安装目录： matlabroot/simulink/src/下找到sfuntmpl_basic.c文件，并以文本的形式打开
- 从模板文件来看，除了必须的文件头和文件尾之外，中间部分代码主要是初始化和其他与S-Function API交互的方法函数。这些函数包括工作向量函数，初始化所需的宏函数，还有一些在模板中没有列出的更多功能的宏函数等。当用户实现的S-Function需要访问这些接口时，可以从simstruct.h头文件中查找相关的宏定义，并从文献Simulink 7 developing S-Function中查找相应的与SimStrut结构有关的函数的定义和用法描述，进而写出实现自己目的的C MEX S-Function代码。
- 初始化所需的宏函数如下表所示

C MEX S-Functon初始化所需宏函数

宏函数定义	功能描述
ssSetNumContStates(S, numContStates)	设置连续状态个数
ssSetNumDiscStates(S, numDiscStates)	设置离散状态个数
ssSetNumOutputs(S, numOutputs)	设置输出个数
ssSetNumInputs(S, numInputs)	设置输入个数
ssSetDirectFeedthrough(S, dirFeedThru)	设置是否存在直接馈通
ssSetNumSampleTimes(S, numSamplesTimes)	设置采样时间的数目
ssSetNumInputArgs(S, numInputArgs)	设置输入参数个数
ssSetNumRWork(S, numIWork)	设置各种工作向量的维数，实际上是为各个工作向量分配内存提供依据
ssSetNumPWork(S, numIWork)	

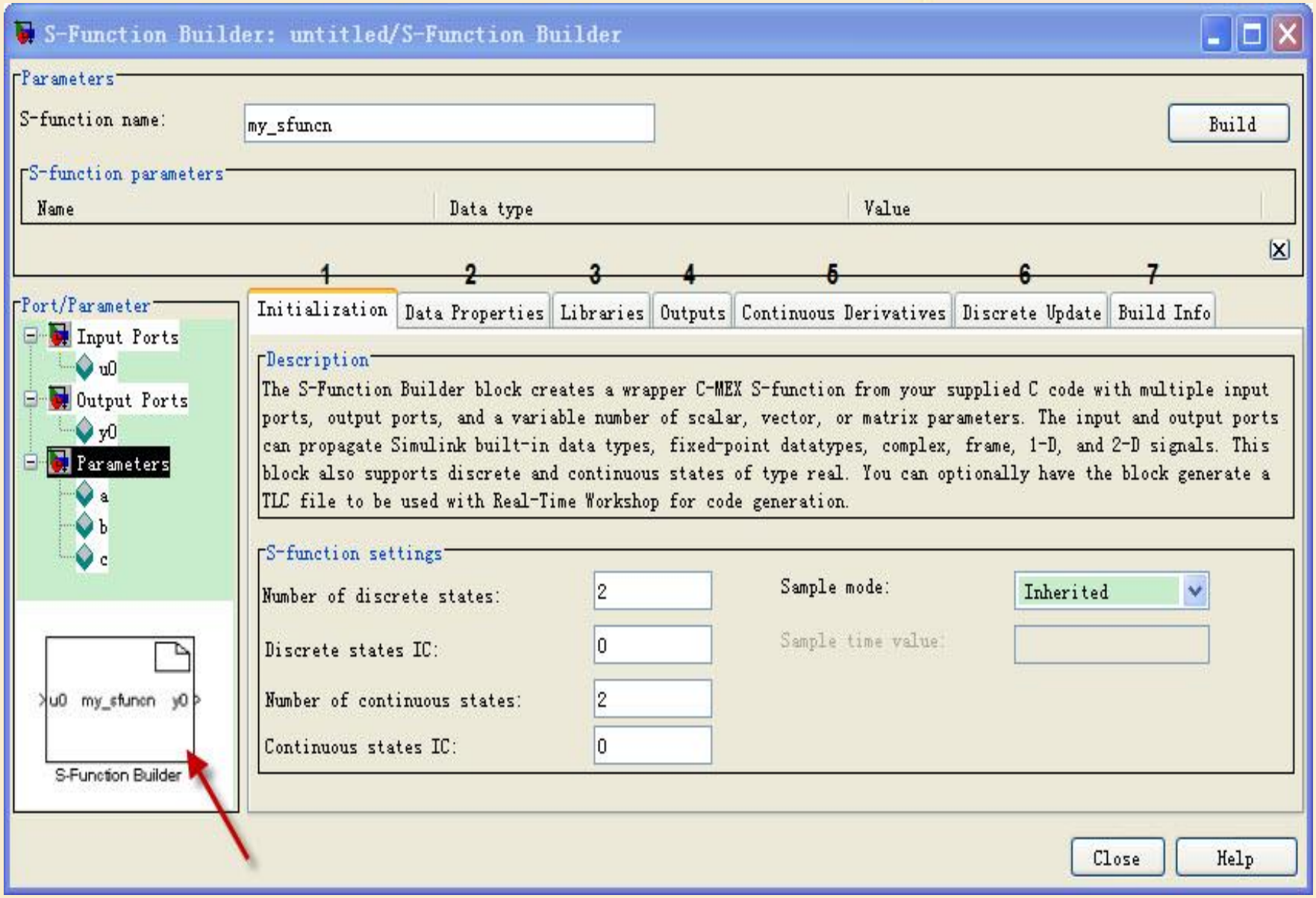
4. C MEX S-Function 包装程序

- 当用户需要将已有的程序、算法集成到Simulink框图模型中时，通常使用S-Function包装程序（MEX S-Function Wrappers）来完成这个任务。所谓的S-Function包装程序就是一个可以调用其它模块代码的S-Function，实际上就是通过S-Function的形式来调用其它语言（MATLAB语言除外）编写的程序。使用外部模块时，需要在S-Function中将已有的代码声明为extern（即外部函数），并且还必须在mdlOutputs方法中调用这些已有的代码
- 编译带有外部函数的S-Function时，需要将已有程序的源文件加在S-Function源文件的后面即可，如下所示：

```
>> mex mysfcn_wrap.c my_wrapfcn.c
```

5. S-Function Builder

- S-function Builder是Simulink为用户编写常用的C MEX S-Function提供的一种方便、快捷的开发工具。使得用户无需了解众多的宏函数就可以编写出自己的S-Function，只要在对应的位置填入所需的信息和代码，S-function Builder就会自动生成C MEX S-Function源文件，并且编译起来也非常方便，只需单击Build按钮，就会生成用户所需要的MEX文件。
- 在Simulink/User-Defined Functions模块库中双击S-function Builder模块图标，即可打开如下图所示的S-function Builder界面。
- 很容易看出1、4、5、6选项卡对应着S-Function的四个最常用的回调方法。打开S-function Builder为用户生成的C源文件，就会发现在各个页面填入的信息和代码被放入了对应的回调函数中 。


S-Function Builder: untitled/S-Function Builder
Parameters
S-function name:
my_sfuncn
Build
S-function parameters
Name
Data type
Value
1
2
3
4
5
6
7
Port/Parameter
Input Ports
u0
Output Ports
y0
Parameters
a
b
c
u0 my_sfuncn y0
S-Function Builder
Initialization
Data Properties
Libraries
Outputs
Continuous Derivatives
Discrete Update
Build Info
Description
The S-Function Builder block creates a wrapper C-MEX S-function from your supplied C code with multiple input ports, output ports, and a variable number of scalar, vector, or matrix parameters. The input and output ports can propagate Simulink built-in data types, fixed-point datatypes, complex, frame, 1-D, and 2-D signals. This block also supports discrete and continuous states of type real. You can optionally have the block generate a TLC file to be used with Real-Time Workshop for code generation.
S-function settings
Number of discrete states:
2
Discrete states IC:
0
Number of continuous states:
2
Continuous states IC:
0
Sample mode:
Inherited
Sample time value:
Close
Help

主要内容

- Simulink简介
- Simulink动态系统建模与仿真
- S-Funcition
- 命令行仿真技术
- 综合实例

第四节 命令行仿真技术

一、命令行方式建模

用户除了使用Simulink以图形方式建立动态系统模型之外，也可以使用命令行方式进行系统建模，然后进行动态系统的仿真和分析。

命令行方式建模基本上又回到了写程序的轨道上来，即是借助MATLAB/Smulink提供的函数对模块进行操作，连接、建模，设置参数并进行仿真。相对于图形化编程，显得费时费力。但对于习惯写代码的软件用户来说，也不啻为一种有效的解决问题的方式。

Simulink中常用的系统建立命令见下表

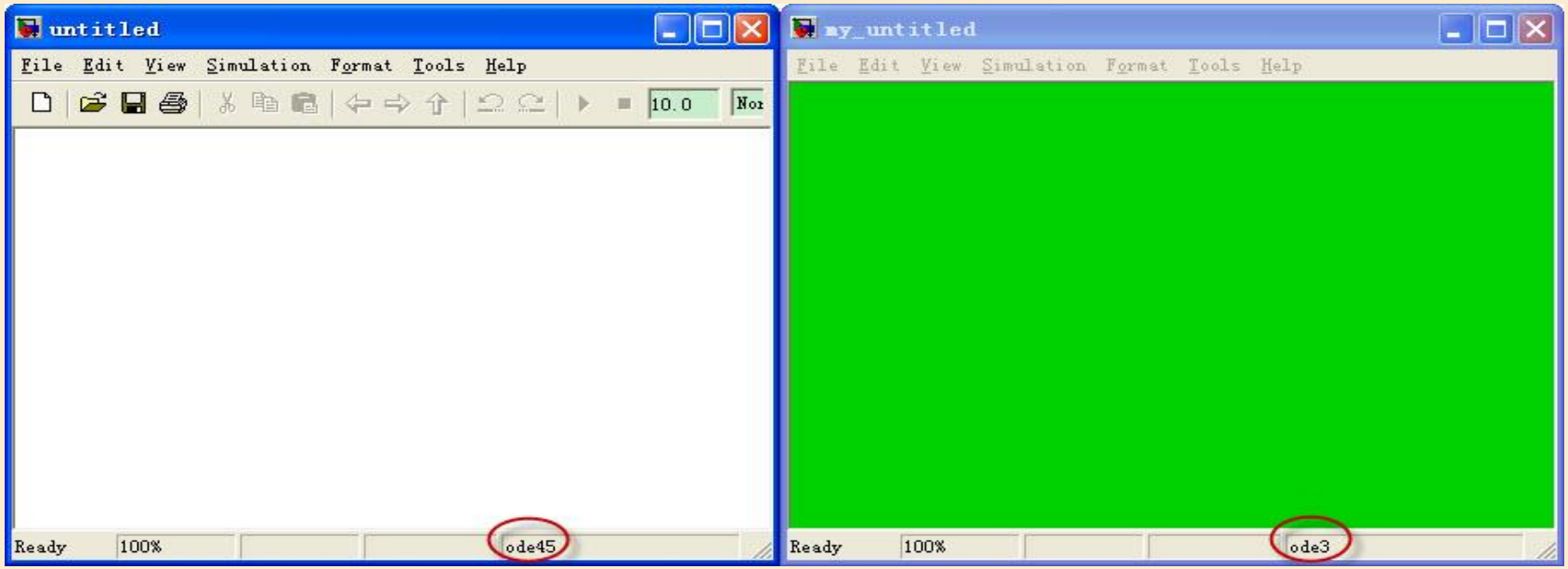
命 令	功 能
New_system	建立一个新的Simulink系统模型
Open_system	打开一个已存在的Simulink系统模型
Close_system, bdclose	关闭一个Simulink系统模型
Save_system	保存一个Simulink系统模型
Find_system	查找Simulink系统模型、模块、连线及注释
Add_block	在Simulink系统模型中加入指定模块
Delete_block	在Simulink系统模型中删除指定模块
Replace_block	在Simulink系统模型中替代指定模块
Add_line	在Simulink系统模型中加入指定连线
Delete_line	在Simulink系统模型中删除指定连线
Get_param	获取Simulink系统模型中的参数
Set_param	设置Simulink系统模型中的参数
Gcb	获得当前模块的路径名
Gcs	获得当前系统模型的路径名
Gcbh	获得当前模块的操作句柄
Bdroot	获得最上层系统模型的名称
Simulink	打开Simulink的模块库浏览器

Simulink的基本菜单和工具栏命令如New，Open和Save等操作都是通过上表中的命令来实现的

```
function new_model(modelname)
% NEW_MODEL('MODELNAME') 创建一个名为'MODELNAME'的新模型
% 没有'MODELNAME'参数，新模型默认命令为'my_untitled'，
（Simulink系统默认为'untitled'）
if nargin == 0
modelname = 'my_untitled';                          % 自己定制默认的模型名
end
open_system(new_system(modelname));                 % 创建和打开一个模型
set_param(modelname, 'ScreenColor', 'green');       % 设置缺省屏幕颜色
set_param(modelname, 'Solver', 'ode3');             % 设置默认求解器
set_param(modelname, 'Toolbar', 'off');             % 设置缺省工具栏的可见性
save_system(modelname);                             % 保存模型
```

- 将这段代码保存为当前文件夹内的new_model.m文件。然后用默认的系统配置新建一个模型，在MATLAB命令行下输入如下命令，也会产生一个用上面函数定制的新建模型

```
>> new_model
```



- 从上图可以看到，系统默认生成名为untitled的空白模型，屏幕颜色为白色，工具栏可见，求解器为变步长ode45。而从上面函数运行后产生的是名为my_untitled的空白模型，屏幕颜色为绿色，工具栏不可见，求解器为定步长ode3。

二、命令行方式仿真

1. sim 和 set_ param 命令

sim命令的基本语法格式如下：

simout = sim('*model*',*Parameters*)

[t,x,y] = sim('*model*',timespan,options,ut)

[t,x,y1,y2,…,yn] = sim('*model*',timespan,options,ut)

- 上述sim命令格式中，第一行是单输出格式，参数*model*是系统模型的名称，也就是去掉扩展名.mdl的文件名。*parameters*则是参数名称-参数值对的列表。simout是一个包含所有仿真输出（日志时间，状态和信号）的单一的Simulink.SimulationOutput对象。
- 第2，3行是多输出格式。其中除*model*之外的各参数说明如下

Timespan：指定系统仿真时间范围，可以为下面几种形式：

tFinal：设置仿真终止时间，仿真起始时间默认为0

[tStart tFinal]：设置仿真起始时间和终止时间

[tStart OutputTimes tFinal]：设置仿真起、止时间，并设置仿真返回的时间向量[tStart OutputTimes tFinal]，其中tStart, OutputTimes, tFinal必须按照升序排列

options：由simset命令设置的除仿真时间之外的仿真参数，为一结构体变量。这样设置的仿真参数将覆盖模型默认的参数。

ut：表示系统模型顶层的外部可选输入。ut可以是MATLAB函数，可以使用多个外部输入ut1，ut2….

t：返回系统仿真时间向量

x：返回系统仿真状态变量矩阵。首先是连续状态，然后是离散状态

y：返回系统仿真的输出矩阵。按照顶层输出Outport模块的顺序输出，如果输出信号为向量输出，则输出信号具有与此向量相同的维数

y1,…,yn：返回多个系统仿真的输出

- 上述各参数中，除*model*外，其他的仿真参数设置均可以取值为空矩阵，此时sim命令对没有设置的仿真参数使用默认的参数值进行仿真。默认的参数值由系统模型框图所决定。
- 如果用户对连续系统进行仿真，必须设置合适的仿真求解器，因为默认的仿真求解器为变步长ode45求解器。
- 使用sim命令进行仿真后，用户还可以使用set_param命令来开始、停止、暂停或继续这个仿真过程，达到更新模型或者将所有的数据日志变量写到基工作空间。

set_param命令的基本语法格式如下：

set_param('*model*','SimulationCommand', '*cmd*')

- 其中，'*model*' 是系统模型的名称，'*cmd*' 取值可以是start，stop，pause，continue，updatea或WriteDataLogs中的任何一个。
- 类似的，用户还可以使用get_param命令来检查仿真状态，其格式如下：

  get_param('*model*','SimulationStatus')
- 执行后，会返回stopped，initializing，running，paused，updating，terminating或external等值。
- S-Function也能够用set_param命令控制仿真执行。C MEX S-Function调用set_param命令时需要使用宏mexCallMATLAB。

※ 注意：用户如果使用matlab –nodisplay启动开始一个会话，就不能够使用set_param命令来运行仿真会话。

2. simget 和 simset 命令

simget命令获得表示系统仿真参数的结构体变量，该命令语法格式如下：

struct = simget('*model*')

value = simget('*model*','param')

value = simget(OptionStructure, param)

- 上述simget命令格式中，参数'*model*' 意义同set_param。第一行用法是为了获得指定系统模型的所有仿真参数设置的结构体变量，第二行是为了获得指定系统模型中指定仿真参数param的取值。第三行则是为了获得系统仿真参数选项中指定的仿真参数property的取值。
- 上述第一行命令将获得已经打开的系统模型*model*的仿真参数选项struct，此为一结构体变量，包含一些常用内容，还有大量不常设置的内容，这里不予介绍，读者可以看书

- 使用simset命令就可对options中的各结构变量进行操作，修改设置。simset命令的语法格式如下：

options = simset(property, value,...)

options = simset(old_opstruct,property,value,...)

options = simset(old_opstruct,new_opstruct)

- 第一条是设置指定的仿真参数property选项的值为value
- 第二条是修改仿真参数结构体变量中已经存在的指定仿真参数选项。
- 第三条则是合并已经存在的两个仿真参数结构体变量，并且使用new_opstruct中的域值覆盖old_opstruct中具有相同域或的域值。只用simset不加参数，则显示所有的仿真参数选项及其可能的取值。

3. simplot 命令

在观测动态系统仿真结果时，Simulink/Sinks模块库中的Scope模块是最常用的模块之一。它可以使用户在类似示波器的图形界面中观测系统仿真结果的输出，而非使用诸如plot绘制系统的输出。使用Scope最重要的优点是可以通过对Scope模块的操作，对系统输出进行方便的观测，这一点是plot等命令所不及的。

simplot命令可以绘制出与使用Scope模块相类似的图形来输出动态系统的仿真结果，simplot命令的语法格式如下：

simplot(data)

simplot(time,data)

- 参数data是动态系统仿真结果的输出数据，一般由Outport模块，To Workspace模块等产生的输出。其数据类型可以为矩阵、向量或是结构体等。
- 参数time是动态系统仿真结果的输出时间向量。当系统输出数据为带有时间向量的结构体变量时，此参数被忽略。

第五节 综合实例

受篇幅所限，所有的实例和细节均未给出，请读者仔细阅读例子，并结合程序代码加以理解。

请谅解 !!! 谢谢!!!

例11-2 利用Simulink仿真曲线。

仿真过程如下：

(1) 启动Simulink并打开模型编辑窗口。

(2) 将所需模块添加到模型中。

(3) 设置模块参数并连接各个模块组成仿真模型。

设置模块参数后，用连线将各个模块连接起来组成仿真模型。

(4) 设置系统仿真参数。

(5) 开始系统仿真。

(6) 观察仿真结果。

例11-3 利用Simulink仿真求定积分。

仿真过程如下：

(1) 打开一个模型编辑窗口。

(2) 将所需模块添加到模型中。

(3) 设置模块参数并连接各个模块组成仿真模型。

(4) 设置系统仿真参数。

(5) 开始系统仿真。

(6) 观察仿真结果。

11.3.3 系统仿真实例

至此，可以总结出利用Simulink进行系统仿真的步骤：

(1) 建立系统仿真模型，这包括添加模块、设置模块参数以及进行模块连接等操作。

(2) 设置仿真参数。

(3) 启动仿真并分析仿真结果。

例11-4 有初始状态为0的二阶微分方程

x''+0.2x'+0.4x=0.2u (t), 其中u(t)是单位阶跃函数，试建立系统模型并仿真。

方法1：用积分器直接构造求解微分方程的模型。

方法2：利用传递函数模块建模。

方法3：利用状态方程模块建模。

11.4 使用命令操作对系统进行仿真

从命令窗口运行仿真的函数有4个，即sim、simset、simget和set_param。

1．sim函数

sim函数的作用是运行一个由Simulink建立的模型，其调用格式为：

```
[t, x, y ] =sim(modname,timespan,options,data);
```

2．simset函数

simset 函数用来为sim函数建立或编辑仿真参数或规定算法，并把设置结果保存在一个结构变量中。它有如下4种用法：

(1) options=simset(property,value,…)：把property代表的参数赋值为value，结果保存在结构options中。

(2) options=simset(old_opstruct,property,value,…)：把已有的结构old_opstruct(由simset产生)中的参数property重新赋值为value，结果保存在新结构options中。

(3) options=simset(old_opstruct,new_opstruct)：用结构new_opstruct的值替代已经存在的结构old_opstruct

3．simget函数

simget函数用来获得模型的参数设置值。如果参数值是用一个变量名定义的，simget返回的也是该变量的值而不是变量名。如果该变量在工作空间中不存在(即变量未被赋值)，则Simulink给出一个出错信息。

该函数有如下3种用法：

(1) struct=simget(modname)：返回指定模型model的参数设置的options结构。

(2) value=simget(modname,property)：返回指定模型model的参数property的值。

(3) value=simget(options,property)：获取options结构中的参数property的值。如果在该结构中未指定该参数，则返回一个空阵。

4．set_param函数

set_param函数的功能很多，这里只介绍如何用set_param函数设置Simulink仿真参数以及如何开始、暂停、终止仿真进程或者更新显示一个仿真模型。

(1) 设置仿真参数

调用格式为：

set_param(modname,property,value,…)

其中modname为设置的模型名，property为要设置的参数，value是设置值。这里设置的参数可以有很多种，而且和用simset设置的内容不尽相同，相关参数的设置可以参考有关资料。

(2) 控制仿真进程

调用格式为：

set_param(modname,'SimulationCommand','cmd')

其中mode为仿真模型名称，而cmd是控制仿真进程的各个命令，包括start、stop、pause、comtinue或update。

在使用这两个函数的时候，需要注意必须先把模型打开。

11.5 子系统及其封装技术

11.5.1 子系统的建立

建立子系统有两种方法：通过Subsystem模块建立子系统和通过已有的模块建立子系统。两者的区别是：前者先建立子系统，再为其添加功能模块；后者先选择模块，再建立子系统。

1．通过Subsystem模块建立子系统

操作步骤为：

(1) 先打开Simulink模块库浏览器，新建一个仿真模型。

(2) 打开Simulink模块库中的Ports & Subsystems模块库，将Subsystem模块添加到模型编辑窗口中。

(3) 双击Subsystem模块打开一个空白的Subsystem窗口，将要组合的模块添加到该窗口中，另外还要根据需要添加输入模块和输出模块，表示子系统的输入端口和输出端口。这样，一个子系统就建好了。

2．通过已有的模块建立子系统

操作步骤为：

(1) 先选择要建立子系统的模块，不包括输入端口和输出端口。

(2) 选择模型编辑窗口Edit菜单中的Create Subsystem命令，这样，子系统就建好了。在这种情况下，系统会自动把输入模块和输出模块添加到子系统中，并把原来的模块变为子系统的图标。

例11-5 PID控制器是在自动控制中经常使用的模块，试建立PID控制器的模型并建立子系统。

步骤如下：

(1) 先建立PID控制器的模型。

(2) 建立子系统。

11.5.2 子系统的条件执行

1．使能子系统

建立使能子系统的方法是：打开Simulink模块库中的Ports & Subsystems模块库，将Enable模块复制到子系统模型中，则系统的图标发生了变化。

例11-6 利用使能子系统构成一个正弦半波整流器。

操作步骤如下：

(1) 打开Simulink模块库浏览器并新建一个仿真模型。

(2) 将Sine Wave、Enabled Subsystem、Scope 3个模块拖至新打开的模型编辑窗口，连接各模块并存盘。其中使能信号端接Sine Wave模块。

为了便于比较，除显示半波整流波形外，还显示正弦波，故在示波器属性窗口将Number of axes设置为2。

使能子系统建立好后，可对Enable模块进行参数设置。

(3) 选择Simulink菜单中的Start命令，就可看到半波整流波形和正弦波形。

2．触发子系统

触发子系统是指当触发事件发生时开始执行子系统。与使能子系统相类似，触发子系统的建立要把Ports & Subsystems模块库中的Trigger模块添加到子系统中或直接选择Triggered Subsystem模块来建立触发子系统。

例11-7 利用触发子系统将一锯齿波转换成方波。

操作步骤如下：

(1) 用Signal Generator、Triggered Subsystem和Scope模块构成子系统。

(2) 选择Simulink菜单中的Start命令，就可看到波形。

3．使能加触发子系统

所谓使能加触发子系统就是把Enable和Tirgger模块都加到子系统中，使能控制信号和触发控制信号共同作用子系统的执行，也就是前两种子系统的综合。该系统的行为方式与触发子系统相似，但只有当使能信号为正时，触发事件才起作用。

11.5.3 子系统的封装

所谓子系统的封装(Masking)，就是为子系统定制对话框和图标，使子系统本身有一个独立的操作界面，把子系统中的各模块的参数对话框合成一个参数设置对话框，在使用时不必打开每个模块进行参数设置，这样使子系统的使用更加方便。

子系统的封装过程很简单，先选中所要封装的子系统，再选择模型编辑窗口Edit菜单中的Mask subsystem命令，这时将出现封装编辑器(Mask Editor)对话框。

Mask Editor对话框中共包括4个选项卡：Icon、Parameters、Initialization和Documentation。子系统的封装主要就是对这4页参数进行设置。

11.6 S函数的设计与应用

S函数称为系统函数(System Function)，它有固定的程序格式。用MATLAB语言可以编写S函数，此外还可以采用C、C++、FORTRAN和Ada等语言编写。

11.6.1 用MATLAB语言编写S函数

编写S函数有一套固定的规则，为此，Simulink提供了一个用M文件编写S函数的模板。该模板程序存放在toolbox\simulink\blocks目录下，文件名为sfuntmpl.m。用户可以从这个模板出发构建自己的S函数。

1．主程序

S函数主程序的引导语句为：

function [sys,x0,str,ts]=fname(t, x, u, flag)

2．子程序

S函数M文件共有6个子程序，供Simulink在仿真的不同阶段调用。

11.6.2 S函数的应用

例11-8 采用S函数实现模块y=nx，即模块的功能是把一个输入信号n倍以后再输出。

(1) 利用MATLAB语言M文件。

(2) 模块的封装与测试。